

Journal Pre-proof

Logic-based Benders decomposition combined with column generation
for mobile 3D printer scheduling problem

Tao Li, Hu Qin, Nan Huang

PII: S0305-0483(25)00168-9
DOI: <https://doi.org/10.1016/j.omega.2025.103442>
Reference: OME 103442

To appear in: *Omega*

Received date : 14 April 2025

Accepted date : 10 October 2025



Please cite this article as: T. Li, H. Qin and N. Huang, Logic-based Benders decomposition combined with column generation for mobile 3D printer scheduling problem. *Omega* (2025), doi: <https://doi.org/10.1016/j.omega.2025.103442>.

This is a PDF file of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability, but it is not yet the definitive version of record. This version will undergo additional copyediting, typesetting and review before it is published in its final form, but we are providing this version to give early visibility of the article. Please note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

© 2025 Published by Elsevier Ltd.

Logic-based Benders Decomposition Combined with Column Generation for Mobile 3D Printer Scheduling Problem

Tao Li¹, Hu Qin¹, Nan Huang^{*,2}

Abstract

The principles of sharing become increasingly important for boosting business profitability. In the realm of the sharing economy, 3D printers possess the potential to meet the printing demands of an expanded customer base. This paper investigates a class of mobile 3D printer scheduling problems that consider various aspects such as printer allocation, transportation, and production. 3D printers can be transported using truck between customer locations to efficiently fulfill customer orders and achieve optimal resource allocation. A mixed-integer linear programming model is proposed to describe this problem. After analyzing the characteristics and structure of the model, a logic-based Benders decomposition algorithm framework is designed for solving this problem. To address the specific characteristics of the Benders sub-problem, we define a novel tandem sequence structure and develop a tandem-sequence-based column generation for solving the Benders sub-problem. Three strategies, namely dominance rules, effective upper bounds, and tandem sequence deduplication, are constructed to accelerate the algorithm's convergence. To evaluate the performance of the proposed algorithm framework and acceleration strategies, a comprehensive set of 320 instances and a real-world set of 14 instances are generated for rigorous testing. The experimental results affirm the effectiveness of these approaches and algorithm.

Keywords:

Mobile 3D printer schedule, Logic-based Benders decomposition, Column generation, Tandem sequence

*Corresponding author. Email: nan_huang@hotmail.com

¹School of Management, Huazhong University of Science and Technology, Wuhan 430074, China

²College of Science and Engineering, Hamad Bin Khalifa University, Doha, Qatar

1. Introduction

As a newly emerging economic model in recent years, the core concept of the *Shared Economy (SE)* is to build online sharing platforms, where suppliers and customers can offer services or upload demands, aiming to achieve efficient utilization of idle resources through process matching (Hossain, 2020; Benjaafar and Hu, 2020; Mont et al., 2020; Klarin and Suseno, 2021). Propelled by social manufacturing and industry advancements (Jiang et al., 2016), the shared economy has gained widespread adoption across diverse industry sectors, encompassing transportation services (e.g., Uber (2024), Didi (2024)), bike-sharing initiatives (e.g., Ofo (2014)), and more. This collaborative model can facilitate the efficient allocations of societal resources. Meanwhile, it can also present substantial business prospects and economic advantages for enterprises.

Building on this, a growing number of suppliers are now contemplating sharing their resources in order to increase profits, particularly in scenarios with limited and costly resources, as well as geographically scattered customers. Trucks or other transportation methods are employed to deliver processing machines and other resources to customers' production sites (Jackson et al., 2008; Shahmoradi-Moghadam and Schönberger, 2021). 3D printers, as expensive and limited resources, play a crucial role in multiple industries (such as printing medical prosthetics, creating architectural models, manufacturing automotive components, etc.). However, due to the scattered demands, processing all orders promptly poses a challenge. To tackle this issue, several 3D printing companies, such as CassaMobile (2013), have begun providing mobile delivery production services. Firstly, the 3D printers are integrated into a shipping container (see Figure 1) and transported or delivered by truck. When the printer is delivered to the customer, the order can begin processing. Once all the orders at the customers' locations are fulfilled, the machines are reloaded onto the truck and transported back to the company.

Although this mobile delivery production mode enhances customers' willingness to make purchases and increases the economic benefits for businesses (Wang et al., 2023), it also places higher demands on machine allocation and truck transportation planning. Firstly, the printing speed and efficiency of 3D printers vary, the number of orders at each customer location differs. Thus, the rational allocation of machines becomes a critical factor that affects the completion time. Secondly, the truck routes planning and sequence schedul-

ing have an impact on the delivery time of machines, indirectly affecting the start time of processing. Additionally, the processing duration of orders will influence the time for the truck to collect the machines. Lastly, different orders may require distinct printing materials and techniques, thus the setup time between consecutive orders needs to be taken into consideration.

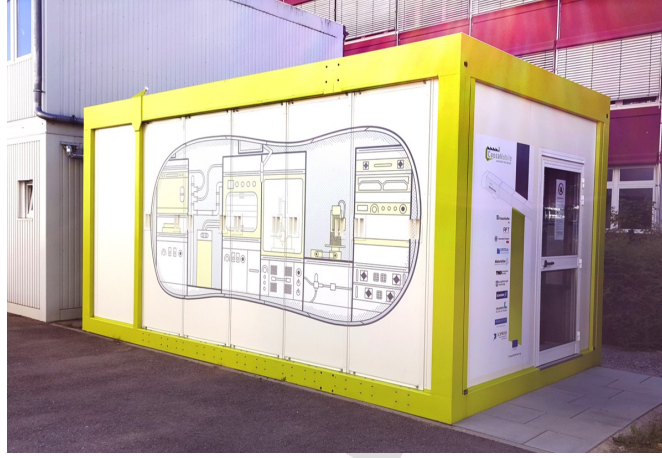


Figure 1: 3D printer container of Cassamobile.

Addressing the challenges and issues faced by businesses mentioned above, this paper explores a mobile 3D printers scheduling problem (M3DPSP). The M3DPSP involves a truck that carries 3D printers departing from a depot and visiting a sequence of customer nodes to deliver an uncertain number of machines for processing. Once the machines complete all processing tasks at a customer node, the truck returns to collect these machines. The collected machines can be either delivered to other customer nodes or directly returned to the depot. Furthermore, the setup time is a crucial constraint to be considered during the operation of 3D printers. Variations in setup times can occur due to the use of different printing materials and techniques for various tasks. Hence, this paper thoroughly addresses a novel M3DPSP with setup times (M3DPSPST) by taking these factors into account.

Given the intricate nature of M3DPSPST, existing literature lacks effective methods, and no prior research has proposed an approach for solving

problems with similar characteristics. Consequently, we concentrate on addressing this significant issue in this paper. The primary contributions of our research are summarized as follows:

- Motivated by real-world applications and requirements in the 3D printer industry, we introduce a novel M3DPSPST.
- A mixed-integer linear programming model is proposed to describe the problem. Subsequently, the logic-based Benders decomposition method is employed to solve M3DPSPST. The Benders cuts are designed to enhance the efficiency of the solution process, and the effectiveness of these cutting strategies has been mathematically proven through rigorous derivation.
- To solve the Benders sub-problem, a tandem-sequence-based column generation method is proposed. Three strategies—dominance rules, effective upper bounds, and tandem sequence deduplication—are developed to accelerate the algorithmic process.
- Four instance sets are generated to conduct extensive computational experiments. The results demonstrate the effectiveness of the algorithm and its components in solving M3DPSPST instances. Additionally, a sensitivity analysis is performed to provide valuable managerial insights for practitioners.

The remainder of this paper is structured as follows: In Section 2, a literature review related to the M3DPSPST is provided. Section 3 presents the problem and describes the mathematical formulations. Section 4 introduces the main components of the Logic-based Benders decomposition (LBBD) algorithm, including the master problem and the sub-problem. Section 5 presents the detail of tandem-sequence-based column generation for solving sub-problem. In Section 6, we discuss our algorithm's acceleration techniques and implementation features. Section 7 presents the experimental results. Finally, this study is concluded in Section 8, and supplementary materials are listed in the appendices.

2. Literature Review

2.1. Mobile 3D printer scheduling

In recent years, there has been increasing recognition and investment in 3D printers by renowned enterprises as emerging process equipment, includ-

ing General Electric, Airbus, Ford Motor Company, and NASA (Mohr and Khan, 2015; Kucukkoc, 2019). In 2018, Amazon introduced the concept of mobile 3D printers, aiming to save time and money (3ders, 2018). These printers can be integrated into vehicles and delivered directly to customers' doors, resulting in reduced inventory space and labor costs. The topic of mobile 3D printers has gained significant attention and is emerging as a potential new research area (Xu et al., 2021; Kantaros and Piromalis, 2021; Kenger et al., 2021; Jandyal et al., 2022; Tracy et al., 2023).

Alarcon-Gerbier et al. (2022) investigated mobile factories equipped with 3D printing technologies. These factories utilize 3D printers that are embedded in standard containers, allowing for easy transportation and enabling on-site and demand-driven production. The authors developed a mixed-integer program model to address this problem, aiming to minimize total costs. In order to improve the solving efficiency, they employed a heuristic method and implemented various distributed rules. Li et al. (2017) utilized 3D printers for additive manufacturing, involving machines with different specifications. They were the first to define and formulate the problem in the literature, and to propose a mathematical model for it. In their heuristic procedures, they employed the Best-fit and Adapted Best-fit rules. Experimental tests demonstrated that their heuristic approach yielded promising results.

Another related problem is known as the mobile facilities problem, where mobile facilities are deployed to cater to temporally and spatially distributed demands in a particular region (Halper and Raghavan, 2011). This concept finds applications in various domains, ranging from providing cellular telephone coverage to supporting humanitarian relief logistics. Güden and Süral (2014) tackled a location problem involving mobile facilities in a real-life railway construction project. The objective was to determine the optimal number and types of facilities to be located while minimizing the total costs. They proposed two mathematical models and a preprocessing heuristic to address this problem. Computational results obtained from a real case study and randomly generated test instances demonstrated the effectiveness of these methods in handling this problem. Han et al. (2021) conducted research on optimizing a dynamic facility location allocation problem. They focused on a busy farming season that was divided into several time periods, aiming to determine the optimal locations for temporary maintenance stations. However, they did not take into account the transportation between stations during different time periods. The problem was formulated as a mixed integer program and solved using a Benders decomposition algorithm. Lei et al. (2016)

conducted research on a two-stage robust optimization model for the mobile facility fleet sizing and routing problem considering demand uncertainty. In their study, the service was considered to begin when a mobile facility (MF) arrived at a location and end when it departed. In contrast, our study allows the machine to continue operating even after the vehicle has left. To solve the recourse problem, the authors developed a two-level cutting plane-based method and a hybrid algorithm in the inner level that combined heuristic techniques.

Pasha et al. (2020) introduced the concept of “factory-in-a-box”, which involves assembling production modules in containers and transporting them to various customer locations. The authors divided the “factory-in-a-box” concept into two sub-problems: the assignment problem and the transport problem. However, they only focused on studying the second sub-problem, the transport problem, in their paper. In our work, we consider both of these two sub-problems. To achieve global optimality, they utilized CPLEX to solve the model, while for large-scale problem instances, they employed four metaheuristic algorithms to find solutions.

2.2. *Truck-drone delivery problem*

As a popular problem that has emerged in recent years (Cai et al., 2025), the truck-drone delivery or truck-robot delivery problem is also a highly relevant area of this study, where trucks and drones are scheduled and coordinated to serve customers (Liang and Luo, 2022). In truck-drone delivery problems, the service is completed once the goods are delivered to the customer (Liu et al., 2024; Wang et al., 2025; Ramadhan et al., 2025). In our problem, however, after the delivered machine is transported to the customer and the service is completed, the pickup of the machine must still be considered.

For example, Zhang et al. (2024) optimized an integrated drone delivery system that combines truck-drone, drone-only, and truck-only services, using continuous approximation models to minimize total costs. They evaluated impacts of operating characteristics on system performance and drone benefits. Results showed optimal designs often mix drone-only and truck-drone delivery, with truck-drone delivery dominant, and that drone cost savings compared to truck-only delivery were significant but parameter-dependent. Inspired by the application of drones in commercial logistics, Shi et al. (2024) investigated emergency material scheduling via cooperative transport of drones, helicopters, and trucks, considering limited transport capacity,

early post-disaster road conditions, and affected sites accessible only by helicopters or drones due to road restrictions. They formulated the problem as a mixed integer programming model and developed a two-stage heuristic algorithm. However, they stipulated that trucks are unable to wait on-site for drones to complete their services before retrieving them.

Similarly, Tang and van Hoes (2024) focused on the truck-drone routing problem with a single truck and drone, proposing an effective solution algorithm. Specifically, they developed an iterative algorithm to compute dual bounds, drawing on connections between decision diagrams and dynamic programming models used for pricing in branch-and-cut-and-price algorithms. Techniques from decision diagram literature were applied to generate and strengthen novel route relaxations, enabling dual bound calculation without column generation. Zhen et al. (2025) explored a novel truck-drone collaborative on-demand delivery mode with three-step order fulfillment: drone pickup, truck transport, and drone delivery. However, due to capacity and battery constraints, each drone was assumed to serve one order per flight. They built a mixed-integer programming model for multi-route transshipment systems, optimizing drone assignments and launch/landing locations, and designed a branch-and-price algorithm for large-scale instances. Nguyen and Hà (2023) also considered that each drone can serve only one customer in each flight, and they combined multiple drones to form a collective drone (c-drone) capable of transporting heavier items.

2.3. Column generation for parallel machine scheduling problem

Column generation, known for its efficiency and precision, is a widely employed algorithm in parallel machine scheduling problems (Breugem et al., 2022; Zhang et al., 2022; Justkowiak and Pesch, 2023; Shao et al., 2023). To the best of our knowledge, column generation algorithm has shown significant effectiveness in solving parallel machine scheduling problems with the objective function of completion time cost (Alvelos et al., 2016; Van Den Akker et al., 2006)

Ozturk (2020) proposed a column generation based decomposition method for parallel batch scheduling of jobs on identical parallel machines. The objective of their study was to minimize the total flow time, which is defined as the sum of completion times for all processed jobs. They developed a time-indexed column generation model, where each column represented a set of jobs processed in the same batch. Through numerical experiments, they

demonstrated that their model and algorithm could provide high-quality solutions within short computation times. [Alfieri et al. \(2021\)](#) addressed a parallel batching scheduling problem, where jobs are grouped into batches and scheduled on a single machine to minimize the total completion time. They proposed a graph-based model to obtain an effective lower bound using linear programming techniques. To solve this problem, they employed a column generation method embedded within a heuristic branch and price algorithm. Experimental results demonstrated that their algorithm successfully solved all tested instances with 30 jobs and a significant number of instances with 40 jobs, indicating its efficacy in handling this problem. [Druetto et al. \(2023\)](#) considered a more complex variant of the problem: the problem of batching jobs with multiple sizes and multiple incompatible families. Two heuristics were proposed to transition from the continuous to the integer solution of the problem: one based on the price-and-branch heuristic, and the other on a variable rounding procedure. A large number of experiments demonstrated the effectiveness of the algorithm. [Chen and Powell \(1999\)](#) explored a class of problems related to scheduling jobs on identical, uniform, or unrelated parallel machines, aiming to minimize an additive criterion. The additive criterion was a function that depends on the completion time of each individual job. They proposed a column generation approach to solve this problem, in which each column represented a schedule on a specific machine. Computational results demonstrated the promise of this approach, showcasing its capability in effectively solving the problem at hand.

However, column generation algorithms may encounter limitations when applied to parallel machine scheduling problems with the objective function of makespan. Calculating the cost of a column in such cases involves accounting for the maximum completion time, which poses challenges to the algorithm's implementation. Handling the maximum value in the scheduling sequence complicates the column generation process. Alternative approaches or modifications may be required to address this difficulty and achieve optimal solutions for makespan-based parallel machine scheduling problems. In one possible scenario where only one machine is available, [Wiechman and Damodaran \(2015\)](#) investigated the single machine batch scheduling problem with non-identical job processing times, aiming to minimize the makespan. They treated a complete machine schedule as a column, where the makespan of the schedule represented the cost of that column. The results obtained through their proposed approach were compared to previous findings achieved using simulated annealing, a genetic algorithm, and a com-

mercial solver. Xu and Nagi (2013) investigated an identical parallel machine scheduling problem with the objective of minimizing the sum of weighted completion time and makespan. They modified the original problem formulation by employing Dantzig-Wolfe decomposition, dividing it into a master problem and a pricing problem. Each new feasible solution was considered as a new column added to the master problem. The pricing problem was solved by solving a three-index mixed-integer programming (MIP) model. In contrast, our paper presents a customized label setting algorithm to search for a new column with negative reduced cost, and the sub-problem model is formulated as a two-index MIP model.

3. Problem Description

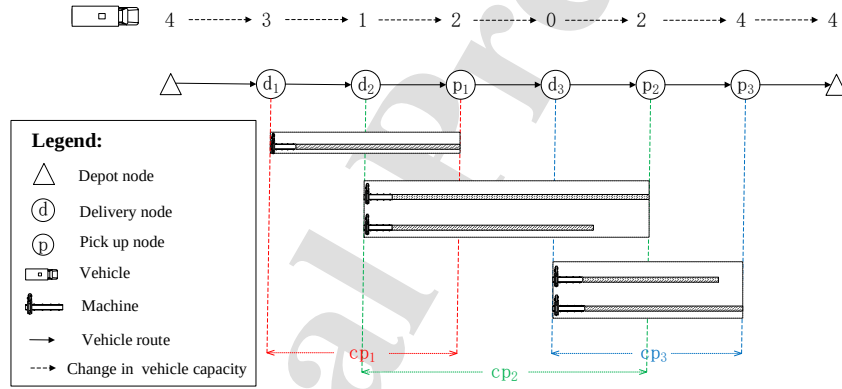


Figure 2: Diagram of the 2-stage problem

This problem considers a two-stage integrated distribution and production problem (Figure 2). The first stage can be regarded as a travel salesman problem with delivery and pick-up (TSPDP). A single vehicle with a capacity of Q departs from depot node to deliver machines to the customer of set S for processing. For modeling purposes, the depot node is decomposed into two dummy nodes: 0^+ , representing the departure node, and 0^- representing the return node. Customer nodes are similarly divided into two sets of dummy nodes, one set representing delivery nodes and the other representing pick-up

nodes. The set S^+ denotes all delivery nodes, while S^- comprises all pick-up nodes. Let n be the total number of customers, then S is defined as the union of these two sets, $S = S^+ \cup S^- = \{1, \dots, n\} \cup \{n+1, \dots, 2n\}$. After processing the last job, the machines at each customer location need to be picked up and transported back to the depot using the same vehicle. Initially, the vehicle departs from the depot with a load of machines. The presence of more machines will enhance the processing efficiency in the second stage. However, if too many machines are delivered to the current customer node, it means that the number of machines delivered to other customer nodes will decrease, which will affect the overall processing efficiency. There is a trade-off between the number of machines and production efficiency. Thus, two decisions need to be made in the first stage: (a). determining the visiting sequence of customer nodes. (b). determining the number of machines assigned to each customer. We define $c_{i,j}$ as the travel time for a vehicle to travel from node i to node j . The service time, which refers to the delivery or pick-up time at customer i , is ignored in our problem.

While the second stage consists of several non-identical parallel machine scheduling problems with setup times (UPMSPST). There is a set of customer requests (jobs to be processed) at each customer i written as C_i ($C_i^0 = C_i \cup \{0\}$, job 0 is a dummy job). Set C_i denotes a series of jobs that need to be processed by the machine assigned in first stage. After all jobs at a customer have finished processing, the same vehicle from the first stage will transport these machines back to the depot. There are two decisions that need to be made in this stage: (a). determining the parallel machines for each job at the customers; (b). sequencing the processing orders on each machine. Let $p_{g,m}$ denote the processing time of job g on machine $m \in M$, where $M = \{1, 2, \dots, u\}$ represents the set of machines in the second stage. Due to the variations between any two jobs, $e_{g,h}$ is used to describe the setup time between job g and job h .

The objective of M3DPSPST is to minimize the total time consumption, which includes both the travel time in the first stage and the machine processing time in the second stage. The coupling of two stages makes solving this problem more challenging. On one hand, the number of machines assigned to each customer, determined in the first stage, affects the production efficiency in the second stage. On the other hand, the makespan in the second stage will subsequently limit the vehicle's serving time in the first stage. To provide a clear description of this problem, we **list all problem assumptions as follows and** propose an arc based formulation in the next section.

- Initially, all customer locations, customer requests, and distances between customer nodes are known.
- The service time, which refers to the machine delivery or pick-up time at the customer node, is ignored.
- The machine can only process one job at a time. Moreover, after delivering the machine, the vehicle can proceed to the next customer or wait for the current one to finish.
- Once all processing tasks are completed, all machines are transported back to the depot by the vehicle.
- Each customer node has sufficient capacity to temporarily house the machines and perform processing tasks.
- Machine preemption is not allowed.

3.1. Mixed-integer Linear Programming Model

A mixed-integer linear programming (MILP) formulation is proposed to describe this problem. In the first stage, let $x_{i,j} = 1$ if the vehicle travels along the first stage arc $(i, j) \in \{(i, j) | i, j \in S \cup \{0^+\} \cup \{0^-\}\}$, and 0 otherwise. Let $z_{i,j} = 1$ if customer node j is visited after node i , which may not necessarily be adjacent to i . Let q_i denote the number of machines delivered or picked up at node $i \in S$. We define $q_i^m = 1$ if machine m is assigned to node i , and 0 otherwise. The load weight (the number of machines) of vehicle when it arrives at node $i \in S \cup \{0\}$ is addressed by v_i . Let t_i be the arrival time of vehicle at node $i \in S$. The decision variables for the second stage are as follows: $y_{g,h,m}^k = 1$ if job h is processed directly after job g on machine m at node k , and 0 otherwise, where $g, h \in C_k, m \in M$. $o_{g,m}$ denotes the completion time of job g on machine m . We use w_i to represent the processing time required for all jobs to be completed at customer node $i \in S$. C_{max} denotes the total time consumption. $\mathcal{M}$ is a big number. All parameters and variables are listed in Tables 1 and 2. Then the MILP model for the problem is as follows:

$$\min \quad C_{max} \tag{1}$$

Table 1: Parameters in MILP.

Parameters	Explanation
Q	Capacity of vehicle
n	Number of customer nodes
$0^+, 0^-$	Departure node and return node
S^+	Set of all delivery customer nodes, $S^+ = \{1, 2, \dots, n\}$
S^-	Set of all pick-up customer nodes, $S^- = \{n+1, n+2, \dots, 2n\}$
S	Set of customer nodes, $S = S^+ \cup S^- = \{1, \dots, n, n+1, \dots, 2n\}$
$c_{i,j}$	Travel time of vehicle from customer node i to j
C_k	Set of customer requests (jobs) at customer node k , ($C_k^0 = C_k \cup \{0\}$)
M	Set of machines, $M = \{1, 2, \dots, u\}$
$p_{g,m}$	Processing time of job g on machine m
$e_{g,h}$	Setup times between job g and job h , and $e_{0,h}$ denotes the initial setup time when h is the first processed job.
$\mathcal{M}$	A big number

Table 2: Decision variables in MILP.

Decision variables	Explanation
$x_{i,j}$	1 if arc (i,j) traveled by the vehicle $(i,j \in S \cup \{0\})$, and 0 otherwise
$z_{i,j}$	1 if customer node j is visited after node i (not necessarily adjacent and $i,j \in S \cup \{0\}$), and 0 otherwise
q_i	Number of machines delivered or picked up at node $i \in S$, $q_i \in \mathbb{Z}$
q_i^m	1 if machine m is assigned to node i ($m \in M, i \in S$), and 0 otherwise
v_i	The load weight of vehicle after serving node i ($i \in S \cup \{0\}$, $v_i \in \mathbb{N}$)
t_i	Arrival time of vehicle at node $i \in S \cup \{0\}$, $t_i \in \mathbb{R}^+$
b_i	The position of node i in the vehicle's visitation sequence, $i \in S \cup \{0\}$, $b_i \in \mathbb{Z}^+$
$y_{g,h,m}^k$	1 if job h is processed directly after job g on machine m at node k ($g,h \in C_k, k \in S, m \in M$), and 0 otherwise
$o_{g,m}$	Completion time of job $g \in C$ on machine $m \in M$
w_i	The processing time required for all jobs to be completed at customer node $i \in S$
C_{max}	Total time consumption

Subject to:

$$\sum_{j \in S \cup 0^-, j \neq i} x_{i,j} = 1, \quad \forall i \in S \quad (2)$$

$$\sum_{i \in S \cup 0^+} x_{i,j} = \sum_{i \in S \cup 0^-} x_{j,i}, \quad \forall j \in S \quad (3)$$

$$\sum_{j \in S} x_{0^+,j} = \sum_{i \in S} x_{i,0^-} \quad (4)$$

$$\sum_{j \in S} x_{0^+,j} = 1 \quad (5)$$

$$v_i \leq Q, \quad \forall i \in S \cup \{0\} \quad (6)$$

$$q_i + q_j = 0, \quad \forall i \in S^+, j \in S^-, j = i + n \quad (7)$$

$$q_i \geq 1, \quad \forall i \in S^+ \quad (8)$$

$$z_{i,j} \geq x_{i,j}, \quad \forall i \in S \cup \{0^+\}, j \in S \cup \{0^-\}, i \neq j \quad (9)$$

$$z_{i,j} \geq 1 + \mathcal{M}(z_{i,k} + z_{k,j} - 2), \quad \forall i \in S \cup \{0^+\}, j \in S \cup \{0^-\}, k \in S, \\ i \neq j, i \neq k, j \neq k \quad (10)$$

$$z_{i+n,j} \geq z_{i,j} + \mathcal{M}(q_i^m + q_j^m - 2), \quad \forall i, j \in S^+, i \neq j, m \in M \quad (11)$$

$$v_0 = v_n = Q \quad (12)$$

$$v_j + \mathcal{M}(1 - x_{i,j}) \geq v_i - q_j, \quad \forall i \in S \cup \{0^+\}, j \in S \cup \{0^-\} \quad (13)$$

$$t_j + \mathcal{M}(1 - z_{i,j}) \geq t_i + c_{i,j}, \quad \forall i \in S \cup \{0^+\}, j \in S \cup \{0^-\} \quad (14)$$

$$t_{i+n} \geq t_i + w_i, \quad \forall i \in S^+ \quad (15)$$

$$\sum_{m \in M} q_i^m = q_i, \quad \forall i \in S^+ \quad (16)$$

$$y_{g,h,m}^k \leq q_k^m, \quad \forall g, h \in C_k^0, k \in S^+, m \in M \quad (17)$$

$$\sum_{m \in M} \sum_{g \in C_k^0, g \neq h} y_{g,h,m}^k = 1, \quad \forall h \in C_k, k \in S^+ \quad (18)$$

$$\sum_{g \in C_k^0} y_{g,h,m}^k = \sum_{g \in C_k^0} y_{h,g,m}^k, \quad \forall h \in C_k, m \in M, k \in S^+ \quad (19)$$

$$\sum_{g \in C_k} y_{0,g,m}^k = \sum_{g \in C_k} y_{g,0,m}^k, \quad \forall m \in M, k \in S^+ \quad (20)$$

$$\sum_{g \in C_k} y_{0,g,m}^k \leq 1, \quad \forall m \in M, k \in S^+ \quad (21)$$

$$o_{h,m} + \mathcal{M}(1 - y_{g,h,m}^k) \geq o_{g,m} + p_{h,m} + e_{g,h}, \\ \forall k \in S^+, g \in C_k^0, h \in C_k, m \in M \quad (22)$$

$$w_k \geq o_{g,m} - t_k, \quad \forall k \in S^+, g \in C_k^0, m \in M \quad (23)$$

$$o_{h,m} + \mathcal{M}(1 - y_{0,h,m}^k) \geq t_k + p_{h,m} + e_{0,h}, \quad \forall k \in S^+, h \in C_k, m \in M \quad (24)$$

$$C_{max} \geq t_{0-} \quad (25)$$

$$x_{i,j} \in \{0, 1\}, \quad \forall i, j \in S \cup \{0\} \quad (26)$$

$$z_{i,j} \in \{0, 1\}, \quad \forall i, j \in S \cup \{0\} \quad (27)$$

$$y_{g,h,m}^k \in \{0, 1\}, \quad \forall g, h \in C_k, k \in S^+, m \in M \quad (28)$$

$$q_i \in \mathbb{Z}, \quad \forall i \in S \cup \{0\} \quad (29)$$

$$q_i^m \in \{0, 1\}, \quad \forall i \in S \cup \{0\}, m \in M \quad (30)$$

$$t_i \geq 0, \quad \forall i \in S \cup \{0\} \quad (31)$$

$$v_i \in \mathbb{N}^+, \quad \forall i \in S \cup \{0\} \quad (32)$$

$$w_i \geq 0, \quad \forall i \in S \quad (33)$$

$$o_{g,m} \geq 0, \quad \forall g \in C_k, m \in M, k \in S \quad (34)$$

The objective function (1) minimizes the total time consumption. Constraints (2) ensure that each customer is visited exactly once. The flow conservation constraints for the first stage are written as constraints (3). Constraints (4) and (5) ensure that the vehicle starts from the depot node and return to the same depot after traveling. The vehicle has a capacity limitation, which is described in constraints (6). Constraints (7) and (8) define the value range of q_i , and the number of machines delivered and picked up

must be equal for the same customer. Constraints (9) and (10) calculate the value of $z_{i,j}$. In constraints (11), a machine cannot be assigned to another customer node before completing processing tasks at the current customer node. Constraint (12) ensures the number of machines transported from the depot is consistent with the number of machines transported back. Constraints (13) calculate the load weight of vehicle after crossing the arc (i, j) . Constraints (14) relate the arrival time between two nodes when a travel arc cross these nodes. For a customer, the processing activities in the second stage cannot start until the machines arrive, and the machines cannot be picked up until these processing activities are completed. These are addressed in constraints (15), which link the first stage with the second stage. Constraints (16) and (17) ensure that the number of machines used is equal to the number of machines assigned at each customer node.

In the second stage, constraints (18) ensure that each job must be processed exactly once. Constraints (19) represent the flow conservation for the second stage. Constraints (20) and (21) ensure that a machine can only process one job at a time. Constraints (22) determine the completion time of job h based on its processing time, the completion time of the previous job, and the setup time. Constraints (23) give the makespan of each machine at each customer node. Constraints (24) link the vehicle's arrival time in the first stage to the machines' departure time if they coincide at a customer node. It denotes that a machine can start processing at a customer only after the freight is delivered to the customer by the vehicle. Constraint (25) calculates the total time. Constraints (26)-(34) are the domain constraints.

The inter-connectivity and coordination between the first stage routing problem and the second stage scheduling problem make the problem highly complicated.

4. Logic-based Benders Decomposition

Through careful investigation and analysis of the MILP formulation, we can find that only constraints (15) and (24) interconnect the first stage with the second stage. By relaxing certain constraints and variables, we can separate the problem into two distinct problems, which can facilitate the problem-solving process. To achieve this, we employ Benders decomposition to split the problem. Firstly, the M3DPSPST is a typical two-stage problem. The first stage involves a vehicle routing problem that considers delivery quantities, while the second stage deals with a parallel machine scheduling problem.

This problem structure is suitable for decomposition using a logic-based Benders decomposition algorithm. Additionally, in the second stage, the variable $y_{g,h,m}^k$ is the primary decision variable. It is evident that $y_{g,h,m}^k$ is a four-index decision variable. After removing the $y_{g,h,m}^k$ variable, the master problem contains only decision variables with at most two indices, significantly speeding up the solution process. Therefore, by removing or reformulating the decision variable $y_{g,h,m}^k$ and its associated constraints, the remaining constraints form the master model. Solving the master problem allows us to obtain the TSPDP solution and the job assignments. In the sub-problem, we need to sequence the jobs assigned to each customer and calculate the value of w_i in each iteration.

According to [Hernández-Pérez and Salazar-González \(2004\)](#), M3DPSPST is NP-hard because it coincides with the traveling salesman problem (TSP) when the vehicle capacity is sufficiently large. In fact, the one-commodity pickup-and-delivery travelling salesman problem (1-PDTSP), which has been proven to be NP-hard by [Mladenović et al. \(2012\)](#), can be considered a special case of M3DPSPST when customer demand and customer service time are fixed. However, in M3DPSPST these two pieces of information (q_i and w_i) are variables. Regarding the sub-problem, according to [Mokotoff \(2004\)](#), when the machines are homogeneous and setup times are not considered, the sub-problem can be viewed as an identical parallel machine scheduling problem. This problem has been proven to be NP-hard, thus making sub-problem NP-hard as well.

In sections 4.1 and 4.2, we propose the formulations of the Benders master problem and sub-problem models.

4.1. Master Problem (MP)

As shown in Figure 2, C_{max} represents the total time from when the vehicle departs from the depot until it returns. The value of C_{max} depends on the travel time in the first stage and the production time in the second stage. For example, in Figure 2, when the vehicle delivers machines to customer d_2 and departs from this node, the remaining machine starts its production task while the vehicle travels to the next destination. There are 3 possible situations to consider when calculating C_{max} : (1). When the vehicle arrives at a pickup node, the machines at that node have just completed their production tasks. In this situation (Figure 3), the travel time from d_2 to p_2 is equal to the processing time cp_2 . When calculating C_{max} , it is acceptable to consider either the travel time or the processing time.

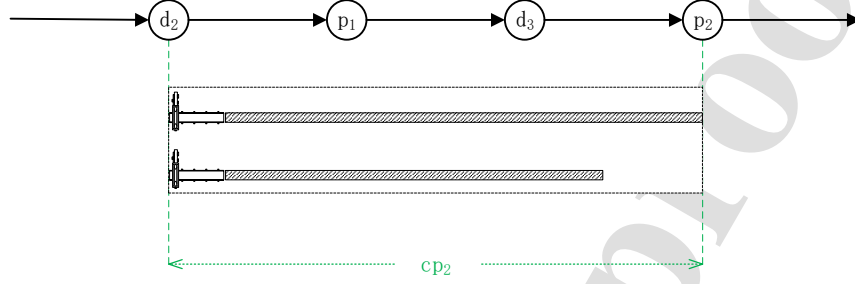


Figure 3: Situation where the travel time is equal to the processing time.

(2). When the vehicle arrives at a pickup node, the machines at that node have already completed the production tasks and waited for a period of t^* . In this situation (Figure 4), the machines complete their production tasks and then wait for a time t^* until the vehicle arrives for pickup. Therefore, the value of C_{max} depends on the travel time, which is equal to the sum of the processing time cp_2 and the waiting time t^* .

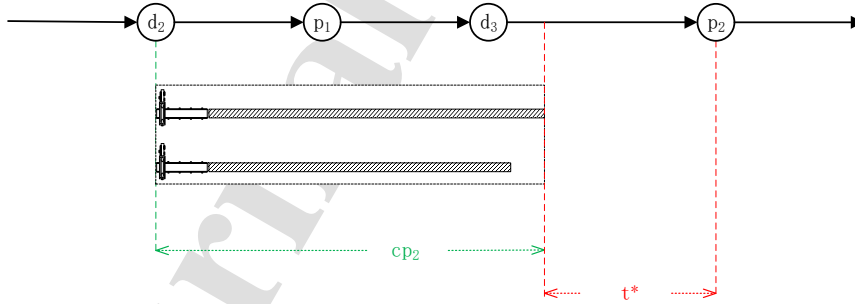


Figure 4: Situation where the travel time is more than the processing time.

(3). When the vehicle arrives at a pickup node, the machines at that node have not yet completed their production tasks, and the vehicle needs to wait

for a period of t^* until the machines can be picked up. In this situation (Figure 5), the node p_2 on the dotted line indicates the actual arrival time of the vehicle, while the node p_2 on the solid line represents the actual departure time from node p_2 . The value of C_{max} is dependent on the processing time cp_2 .

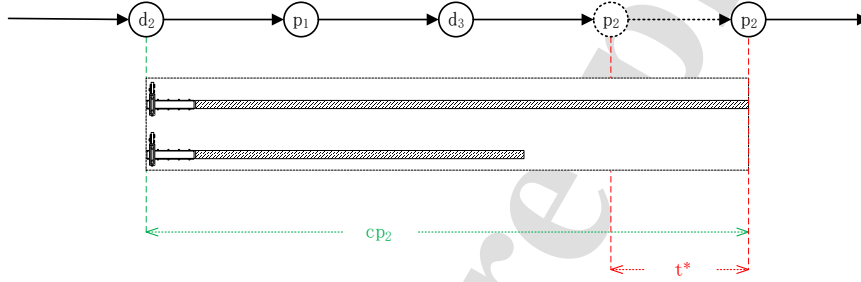


Figure 5: Situation where the travel time is less than the processing time.

Then we propose the master problem (MP) formulation as follows:

$$\min C_{max} \quad (35)$$

Subject to: constraints (2)-(16), (25)-(27), (29)-(33)

$$b_j + \mathcal{M}(1 - x_{i,j}) \geq b_i + 1, \quad \forall i \in S \cup \{0^+\}, j \in S \cup \{0^-\} \quad (36)$$

$$z_{i,i+n} = 1, \quad \forall i \in S^+ \quad (37)$$

$$cuts \quad (38)$$

Where b_i denotes the position of node i in the vehicle's visitation sequence, such that node i is the b_i -th point visited along the vehicle's route. It is important to note that w_i can be 0 in the MP, which may lead to the formation of a vehicle route sub-tour, where $x_{i,i+n} = 1$ and $x_{i+n,i} = 1$. Consequently, constraints (36) are essential to prevent such occurrences. Constraints (37) ensure that the delivery nodes must be visited before the pick-up nodes. In constraints (38), we add new cuts to the MP after each iteration based on

the solutions of the sub-problems. In traditional logic-based Benders decomposition algorithms, these cuts include both feasibility cuts and optimality cuts. However, in our problem, only optimality cuts are present because of constraints (36) and (37). We will discuss them in detail in Section 4.3.

4.2. Sub-Problem (SP)

By solving MP, a set of machines M_k is delivered to each customer node $k \in S^+$. And the SP at node k (SP_k) can be formulated as follows:

$$\min \quad w_k \quad (39)$$

Subject to:

$$y_{g,h,m}^k \leq q_k^m, \quad \forall g, h \in C_k^0, k \in S^+, m \in M_k \quad (40)$$

$$\sum_{m \in M_k} \sum_{g \in C_k^0, g \neq h} y_{g,h,m}^k = 1, \quad \forall h \in C_k, k \in S^+ \quad (41)$$

$$\sum_{g \in C_k} y_{g,h,m}^k = \sum_{g \in C_k} y_{h,g,m}^k, \quad \forall h \in C_k, m \in M_k, k \in S^+ \quad (42)$$

$$\sum_{g \in C_k} y_{0,g,m}^k = \sum_{g \in C_k} y_{g,0,m}^k, \quad \forall m \in M_k, k \in S^+ \quad (43)$$

$$\sum_{g \in C_k} y_{0,g,m}^k \leq 1, \quad \forall m \in M_k, k \in S^+ \quad (44)$$

$$o_{h,m} + \mathcal{M}(1 - y_{g,h,m}^k) \geq o_{g,m} + p_{h,m} + e_{g,h}, \quad \forall k \in S^+, g \in C_k^0, h \in C_k, m \in M_k \quad (45)$$

$$w_k \geq o_{g,m} - t_k, \quad \forall k \in S^+, g \in C_k^0, m \in M_k \quad (46)$$

$$o_{h,m} + \mathcal{M}(1 - y_{0,h,m}^k) \geq t_k + p_{h,m} + e_{0,h}, \quad \forall k \in S^+, h \in C_k, m \in M_k \quad (47)$$

$$y_{g,h,m}^k \in \{0, 1\}, \quad \forall g, h \in C_k, k \in S^+, m \in M_k \quad (48)$$

$$o_{g,m} \geq 0, \quad \forall g \in C_k, m \in M_k \quad (49)$$

Where constraints (40) ensure the assignment of machine m to the customer node k . Constraints (41) ensure that each job at node k must be processed exactly once. Constraints (42) represent the flow conservation.

Constraints (43) and (44) ensure that a machine can only process one job at a time. Constraints (45) determine the completion time of job h based on its processing time, the completion time of the previous job, and the setup time. Constraints (46) provide the makespan of each machine at customer k . Constraints (47) link the vehicle's arrival time in the first stage to the machines' departure time if they coincide at a customer node. Constraints (48) and (49) are domain constraints.

4.3. Logic-based Benders Cuts

In our problem, logic-based Benders cuts are based on variables q_i^m and $x_{i,j}$ by solving the SP. The SP can be handled by a well-designed column generation algorithm, which will be discussed in Section 5 in detail. By solving the MP, we can get a solution where the machines to be loaded onto the vehicle are assigned to each customer node along the vehicle's route. If the MP is unsolvable, it indicates that the instance itself is either unsolvable or infeasible. Otherwise, solving the SPs can yield a corresponding solution. In traditional LBBD, optimality cuts are generated when the SP is feasible, while feasibility cuts are derived when the SP is infeasible. However, in this problem, the infeasibility of the SP can be avoided through some strategies.

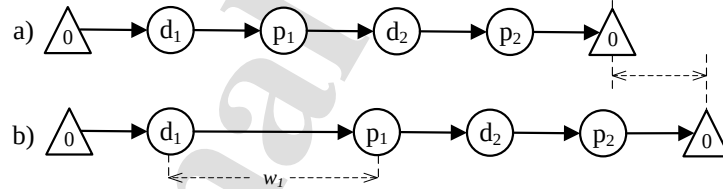


Figure 6: Assume that a) denotes the visit sequence obtained by solving MP. Once the processing duration w_1 is determined through SP, the visit time of node p_1 in scenario b) is correspondingly delayed.

Obviously, the SP is determined by three key variables: q_i^m , $x_{i,j}$ and t_i . The variable $x_{i,j}$ dictates vehicle routing. By incorporating constraints (36) and (37) into the MP, we ensure that all vehicle routes are feasible. The variable q_i^m affects machine allocation and indirectly influences completion time, yet it does not impact the feasibility of SP. In addition, the variable t_i also does not

introduce any feasibility issues. Although the completion time derived from the SP may render the solution of the original master problem infeasible, for example, there could be a scenario where the processing completion time of a particular customer is later than the time when the vehicle leaves after picking up the machines from this customer, thus making the original solution invalid. However, by delaying the time at which the vehicle picks up the machines, a new feasible solution can be generated. Through this process, we can convert an infeasible solution into a feasible one. This new solution adheres to the same driving route and machine allocation scheme and potentially has a greater value than the original one. As illustrated in Figure 6, an increase in completion time at node d_1 may delay arrival time at the customer node p_1 for pick-up machine. However, this scenario does not violate the feasibility constraints. Instead, it merely gives rise to a larger value of the objective function. If the generated solution value is larger than the current best solution, we will consider this type of cuts as optimality cuts and add them to the master problem. Thus, there is no feasibility cuts generation. We focus on generating Benders cuts using only q_i^m and $x_{i,j}$. Then the possible outcomes about an MP solution $S^* = (x^*, z^*, q^*, q^{m*}, v^*, t^*, b^*)$ can be:

- (i) The SP is feasible for q^{m*} , x^* but infeasible for t^* . By converting the solution S^* (as shown in Figure 6), we can obtain the new solution $S' = (x^*, z^*, q^*, q^{m*}, v^*, t', b^*)$ with makespan C'_{max} , which is greater than the makespan C^*_{max} in the MP ($C'_{max} > C^*_{max}$). Then, the following *optimality* cut can be added to the MP:

$$C_{max} \geq C'_{max} - \mathcal{M}f_{q^{m*}, x^*}(q^m, x_{i,j}) \quad (50)$$

where

$$\begin{aligned} f_{q^{m*}, x^*}(q^m, x_{i,j}) = & \sum_{i \in S^+} \sum_{m \in M: q^{m*}=1} (1 - q_i^m) + \sum_{i \in S^+} \sum_{m \in M: q^{m*}=0} q_i^m \\ & + \sum_{i,j \in S: x_{i,j}^*=1} (1 - x_{i,j}) \end{aligned} \quad (51)$$

- (ii) SP is feasible for q^{m*} , x^* and t^* . Consequently, there is no need to convert the solution $S^* = (x^*, z^*, q^*, q^{m*}, v^*, t^*, b^*)$. The makespan C'_{max} is equal to the makespan C^*_{max} within the solution S^* , that is, $C'_{max} = C^*_{max}$. Under these circumstances, this solution serves as an optimal solution for the M3DPSPST problem.

Optimality cut (50) aims to adjust the MP by changing the vehicle routing and the machines assigned to each customer node. By controlling the arcs and the number of machines at customer nodes, we can avoid obtaining the same solution of MP in different iterations as shown in Figure 7.

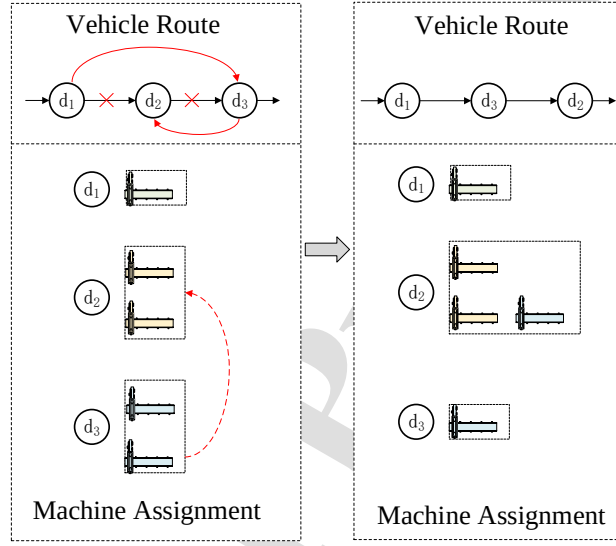


Figure 7: Illustration of logic-based Benders cuts.

The exact algorithm based on the aforementioned logic-based Benders decomposition is outlined as follows. The MP is solved by a commercial solver (such as CPLEX, GUROBI), and the SP is solved by a tailored column generation algorithm discussed in Section 5. The algorithm follows the steps outlined in Figure 8. Where the LB is initialized to 0 and the UB to the maximum finite value of a double-precision floating-point number.

To demonstrate the validity of the cuts, we present the following lemmas regarding the cuts (50).

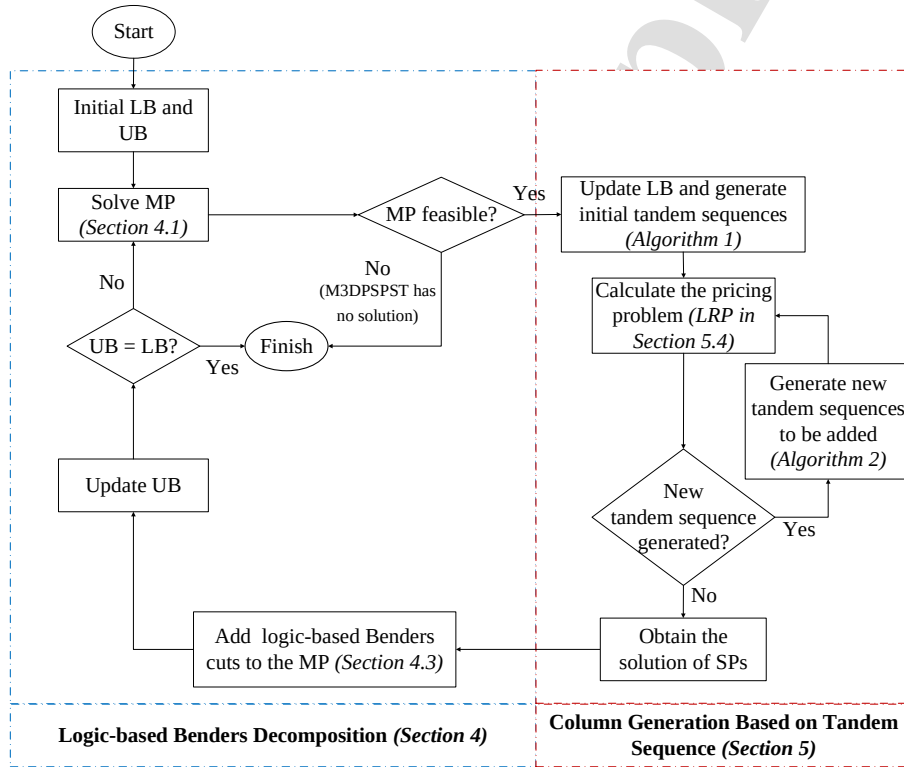


Figure 8: Flow chart of the algorithm.

Lemma 1. *The logic-based Benders cuts (50) satisfy the following properties:*

- (i) *Cuts must remove the current solution from the MP.*
- (ii) $C'_{max} = C'_{max} - \mathcal{M}f_{q^{m^*}, x^*}(q^{m^*}, x^*)$.
- (iii) *Any feasible solution $(\bar{x}, \bar{z}, \bar{q}, \bar{q}^m, \bar{v}, \bar{t}, \bar{b})$ of problem M3DPSPST such that $\bar{q}^m \neq q^{m^*}$ or $\bar{x} \neq x^*$ satisfies $\bar{C}_{max} \geq C'_{max} - \mathcal{M}f_{q^{m^*}, x^*}(\bar{q}^m, \bar{x})$.*

Proof. Properties (i) and (ii) follow directly from the definition of the optimality cut. Regarding property (iii), since $\bar{q}^m \neq q^{m^*}$ or $\bar{x} \neq x^*$ we have $f_{q^{m^*}, x^*}(\bar{q}^m, \bar{x}) \geq 1$, optimality cut (50) can be

$$C'_{max} - \mathcal{M}f_{q^{m^*}, x^*}(q^m, x_{i,j}) \leq C'_{max} - \mathcal{M} \leq 0$$

and the inequality $C_{max} \geq 0$ always holds. $\square$

5. Column Generation Based on Tandem Sequence

In this section, we address the solution approach for the SP problem. The arc-based formulation for SP is complex and challenging to solve. Therefore, we propose a set partitioning model for the SP. In the second stage, we define a feasible schedule for a customer as $l(\mathbf{s}, \mathbf{m})$, where $l \in L$. Here, the schedule l comprehensively encapsulates two critical elements: the job sequence $\mathbf{s}$ and the machine sequence $\mathbf{m}$. Specifically, $\mathbf{s} = \{\mathbf{s}_a\}_{a \in \mathbf{m}}$ denotes a collection of job sequences, where each $\mathbf{s}_a$ represents the sequence of jobs assigned to machine a . Together, these sequences ensure that all jobs are covered, with each individual sequence corresponding to a specific machine in the sequence $\mathbf{m}$. Meanwhile, $\mathbf{m}$ indicates the order in which machines are utilized within the schedule. As illustrated in Figure 9, consider an example of a tandem sequence involving three machines (m_1, m_2, m_3) . Let T_a denote the completion time of machine a , and $\mathbf{s}_a$ represent the sequence of jobs processed on machine a . The job sequence $\mathbf{s}$ can thus be expressed as $\{\mathbf{s}_{m_1}, \mathbf{s}_{m_2}, \mathbf{s}_{m_3}\}$, where $\mathbf{s}_{m_1}$, $\mathbf{s}_{m_2}$, and $\mathbf{s}_{m_3}$ correspond to the job sequences processed on machines m_1 , m_2 , and m_3 , respectively. The machine sequence is given by $\mathbf{m} : (m_1, m_2, m_3)$. This connected sequence is referred to as a *Tandem Sequence*. The objective value of the *Tandem Sequence* is defined as the maximum T_a among all machines.

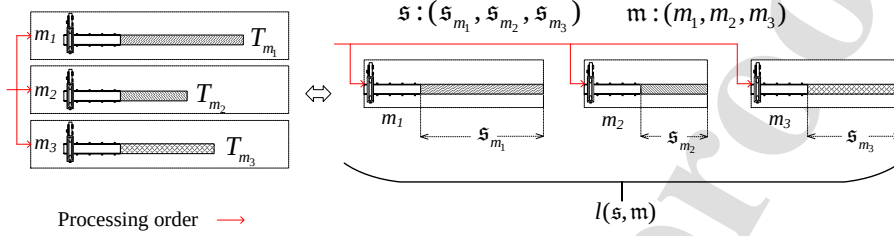


Figure 9: An example of a tandem sequence with 3 machines.

5.1. Tandem Sequence of Machine Schedule

In the second stage, the objective of UPMSPST is to minimize the maximum completion time across all machines at each customer. When there are multiple machines assigned to a customer, we only focus on the machine with the maximum makespan and incorporate it into the objective. The other machines do not need to be taken into account. To address this issue, we propose *Tandem Sequence* that includes all machine schedules for a customer. The generation of this sequence follows the following rules:

- A tandem sequence comprises all parallel machine schedules at a customer node.
- All jobs at a customer node are processed once in a tandem sequence.
- The schedules of parallel machines are connected in a serial structure.
- Tandem sequences start from a machine, and each job is assigned to its preceding machine.

The purpose of the tandem sequence is to facilitate the use of the column generation and labeling algorithm. When dealing with SP at each customer node, multiple jobs need to be scheduled on several parallel machines. The objective of the SP is to minimize the maximum completion time (makespan). However, a challenge arises when using the schedule sequence of each machine as a “column” in the column generation algorithm. Although the completion time for each machine can be computed easily, this value cannot be directly used as the “column” cost in the set partitioning model, as the overall makespan depends on the maximum completion time across all machines. To address this, we treat the combined schedules of all machines (*Tandem*

Sequence) at a customer node as a single “column” in the column generation algorithm. In addition, the tandem sequences allow us to regard machines as special extension points, which is very beneficial for applying the label setting algorithm. We will discuss the process of the label setting algorithm in detail in Section 5.3. To highlight the computational time advantage of tandem sequence-based column, we also compare it with the enumeration method (ENU). For detailed test results, please refer to Appendix E.

5.2. Set Partitioning Model

We introduce R_l as the cost (makespan) of a feasible tandem sequence $l, l \in L_k, k \in S^+$. Let $R_l = \max_{m \in M_k} \{T_m\}$, where M_k is the set of machines delivered to customer node k , and T_m denotes the completion time of machine m . Given a tandem sequence l , we can directly derive two parameters, $\alpha_{l,g}$ and β_l , from $l \in L_k$. Parameter $\alpha_{l,g} = 1$ if job g is served by l , and 0 otherwise. Moreover, β_l represents the number of parallel machines assigned to customer k in tandem sequence l . A binary variable $\gamma_l = 1$ if tandem sequence l is used in the solution, and 0 otherwise. The binary linear programming model for SP of customer node $k \in S^+$ can be formulated as follow:

$$\min \sum_{l \in L_k} R_l \gamma_l \quad (52)$$

Subject to:

$$\sum_{l \in L_k} \alpha_{l,g} \gamma_l = 1, \quad \forall g \in C_k \quad (53)$$

$$\sum_{l \in L_k} \beta_l \gamma_l \leq q_k \quad (54)$$

$$\gamma_l \in \{0, 1\}, \quad \forall l \in L_k \quad (55)$$

The objective function (52) aims to minimize the makespan in the second stage. Constraints (53) ensure that each job is processed by exactly one tandem sequence. Constraints (54) restrict the number of machines used in the second stage to not exceed the number of machines assigned to the customer in the first stage. Constraints (55) define the domain of the variables.

According to the theory of linear programming, we know that a solution to a minimization problem is optimal if the reduced cost of each variable is

nonnegative. In the context of our pricing problem, the reduced cost rc_l of any schedule l can be calculated as follows:

$$rc_l = R_l - \beta_l \lambda_0 - \sum_{g \in C_k} \lambda_g \alpha_{l,g} \quad (56)$$

Here, λ_0 represents the value of the dual variable corresponding to constraint (54), and $\{\lambda_1, \dots, \lambda_g\}$ represent the values of the dual variables corresponding to constraints (53). To tackle the pricing problem, we relax the constraints (55). By solving the relaxed pricing problem, it is possible to identify the machine schedule with negative reduced costs.

Algorithm 1 Greedy Algorithm for Initial Schedule l

- 1: Obtain q_k by solving MP;
 - 2: Initialize C_k and a schedule $l(\mathbf{s}, \mathbf{m})$, $\mathbf{s}$ for job sequence, $\mathbf{m}$ for machine sequence, $\mathbf{s} = \mathbf{m} = \emptyset$;
 - 3: Sort C_k in ascending order of job index;
 - 4: **for** $m = 1, \dots, q_k$ **do**
 - 5: Add dummy job 0 to $\mathbf{s}$, add m to $\mathbf{m}$;
 - 6: **for** $g \in C_k$ **do**
 - 7: **if** $g \bmod q_k = m$ **then**
 - 8: Add job g to $\mathbf{s}$;
 - 9: **end if**
 - 10: **end for**
 - 11: **end for**
 - 12: Obtain the schedule $l(\mathbf{s}, \mathbf{m})$.
-

To effectively initialize the set partitioning model, a simple greedy algorithm is proposed to find an initial solution $l(\mathbf{s}, \mathbf{m})$. Clearly, a feasible initial tandem sequence l must include both the information of job sequence $\mathbf{s}$ and the machine sequence $\mathbf{m}$. First, the jobs in set C_k are sorted in ascending order (see line 3 of Algorithm 1). Then the index of remaining job g divided by the number of machines q_k , and the jobs with same remainder are assigned to the corresponding machine sequence (see lines 6-10 of Algorithm 1). This ultimately yields a feasible initial sequence $l(\mathbf{s}, \mathbf{m})$.

5.3. Label Setting Algorithm

In this section, we present a novel label setting algorithm that incorporates a selection strategy based on rules. The purpose of the label-setting

algorithm is to find all valid tandem sequences, where each tandem sequence must contain all jobs of the customer. A forward partial schedule $l = (\mathbf{s}, \mathbf{m})$ is represented by a label $\mathcal{L}$ associated with the last job g , where $\mathbf{s} = (0, \dots, g)$ and $\mathcal{L} = (c'_g, r_g, \eta_g, \tau_g, \pi_g, \delta_g, N_g, V_g, \Delta_g)$ is composed of the following information:

- c'_g : the reduced cost of the partial schedule l ;
- r_g : the makespan of schedule l ;
- η_g : the last processed job;
- τ_g : the completion time of the last processed job;
- π_g : the current machine;
- δ_g : the number of unused machines;
- N_g : the set of completed processed jobs;
- V_g : the set of next available jobs;
- Δ_g : the set of unused machines.

A label $\mathcal{L}_g$ with $\eta_g = g$ can be extended to a job $h \in C$ or an unused machine yielding one or more new labels based on the following extension functions. Two cases arise when deciding whether to extend to an unused machine:

- (i) If extending to an unused machine, a new label $\mathcal{L}_h$ can be generated, in which the components are computed as follows:

$$\begin{aligned}
 c'_h &= r_h + c'_g - r_g - \lambda_h \alpha_{l,h} \\
 r_h &= \max\{r_g, e_{0,h} + p_{h,\pi_h}\} \\
 \eta_h &= h \\
 \tau_h &= e_{0,h} + p_{h,\pi_h} \\
 \pi_h &= \Delta_g^1 \\
 \delta_h &= \delta_g - 1 \\
 N_h &= N_g \cup \{h\} \\
 V_h &= V_g \setminus \{h\} \\
 \Delta_h &= \Delta_g \setminus \{\Delta_g^1\}
 \end{aligned}$$

where Δ_g^1 is the first member of Δ_g .

- (ii) If extending to a job, a new label $\mathcal{L}_h$ can be generated, in which the components are computed as follows:

$$\begin{aligned} c'_h &= r_h + c'_g - r_g - \lambda_h \alpha_{l,h} \\ r_h &= \max\{r_g, \tau_g + e_{g,h} + p_{h,\pi_g}\} \\ \eta_h &= h \\ \tau_h &= \tau_g + e_{g,h} + p_{h,\pi_g} \\ \pi_h &= \pi_g \\ \delta_h &= \delta_g \\ N_h &= N_g \cup \{h\} \\ V_h &= V_g \setminus \{h\} \\ \Delta_h &= \Delta_g \end{aligned}$$

The above extension equations are calculated sequentially for the variables in the label. In particular, the calculation of the variable τ_h requires consideration of whether an unused machine is extended. When an unused machine is extended, it is equivalent to introducing a new non-identical parallel machine, and τ_h needs to be reset to zero before recalculating the completion time. When no unused machine is extended, the completion time of job h is calculated based on the current time. It is important to note that the calculation of the maximum completion time r_h involves comparing the completion time of the current job with the maximum completion time of the previous label, and selecting the larger value as the final completion time.

In our label setting algorithm, a label represents a state associated with a partial schedule at a certain job. Extension functions above are used to create new schedule. Let $M_k = \{m | m \in M, q_m^k = 1\}$ represent the set of machines delivered to customer node k , and *null* indicates undefined. Let $\mathbb{L}(g) = \{\mathcal{L}_g | \eta_g = g\}$ represent the set of all labels extended to job g . $\mathcal{U}$ is used to temporarily store labels. Firstly, an initial label $\mathcal{L}_0$ is constructed and added to both $\mathbb{L}_0$ and $\mathcal{U}$. Next, if choosing to extend to an unused machine, the corresponding label is generated using the case (i) extension equations and added to the specified set (see lines 6-11 of Algorithm 2). If extend to a job, the V_g of the current label is traversed sequentially, and the state values are updated based on the case (ii) extension equations. Then label dominance testing is required, non-dominated labels are added to the

Algorithm 2 Label Setting Algorithm

```

1: Generate an initial label  $\mathcal{L}_0 = \{0, 0, null, 0, null, q_k, \emptyset, C_k, M_k\}$ , initialize
    $\mathbb{L}(g) = \{\emptyset\}$ , initial  $\mathcal{U} = \emptyset$  for temporary storage;
2: Add label  $\mathcal{L}_0$  to set  $\mathbb{L}(0)$ ,  $\mathcal{U} = \{\mathcal{L}_0\}$ 
3: while  $\mathcal{U}$  is not empty do
4:    $\mathcal{L}'$ : the first label in  $\mathcal{U}$ 
5:    $\mathcal{U} = \mathcal{U} \setminus \mathcal{L}'$ 
6:   //Case (i)
7:   if Extend to an unused machine then
8:     Extend  $\mathcal{L}'$  to 0, generate a new label  $\mathcal{L}'_0$ ;
9:     Update corresponding state values;
10:    Add  $\mathcal{L}'_0$  to  $\mathcal{U}$  and  $\mathbb{L}(0)$ ;
11:   end if
12:   //Case (ii)
13:   if Extend to a job then
14:     for job  $h \in V_h(\mathcal{L}')$  do
15:       if  $\mathcal{L}'$  can be extended to  $h$  then
16:         Extend  $\mathcal{L}'$  to  $h$ , generate a new label  $\mathcal{L}'_h$ ;
17:         Update corresponding state values;
18:         if  $\mathcal{L}'_h$  is not dominated by any label in  $\mathbb{L}(h)$  then
19:           Add  $\mathcal{L}'_h$  to  $\mathcal{U}$  and  $\mathbb{L}(h)$ ;
20:         end if
21:       end if
22:     end for
23:   end if
24: end while
25: Obtain  $\mathbb{L}$ .

```

appropriate set (see lines 12-23 of Algorithm 2). The process from lines 3-24 of Algorithm 2 is repeated until all labels in $\mathcal{U}$ are processed, yielding the complete feasible label set $\mathbb{L}$. Essentially, the label setting method is a dynamic programming approach. Its dominance rule helps the algorithm prune and reduce the search space. We will detail this in sections 6.

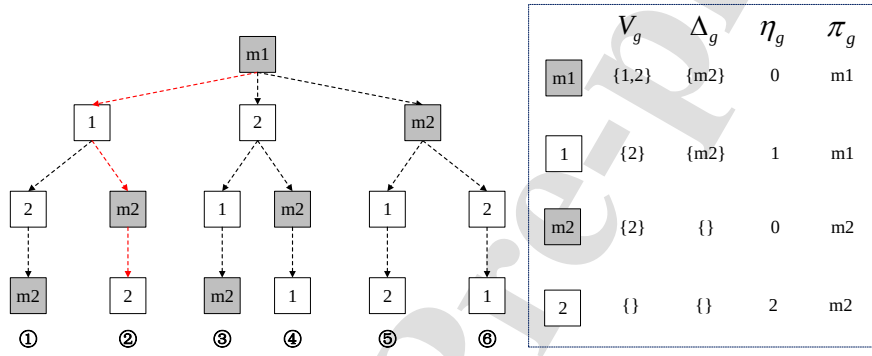


Figure 10: An example of label updating.

Figure 10 illustrates a label update example starting from machine m1. The order in sets V_g and Δ_g is arranged in ascending order of job index and machines index. As can be seen from the label at m1, the next extension of m1 can be job 1, job 2 and machine m2 (that is, $V_g = \{1, 2\}$, $\Delta_g = \{m2\}$). The unused set of machine is $\Delta_g = \{m2\}$, the last processed job is $\eta_g = 0$, and the current machine is m1. Each extension needs to update the corresponding labels according to the extension functions. For instance, in case ②, when extending to job 1, V_g needs to remove the currently visited job 1. When expanding to machine m2, Δ_g needs to remove the new machine m2. The extension stops when both labels V_g and Δ_g are empty sets. By recording the parent node of each label, the complete job and machine sequence can be deduced backwards from the label at job 2.

5.4. Obtain Integer Solutions

To use the column generation algorithm, we relax $\gamma_l \geq 0, \forall l \in L_k$, and obtain the linear relaxation problem (LRP) of the set partitioning model.

According to Baldacci et al. (2008), there always exists an optimal integer solution such that all columns have reduced costs no greater than $SP_{ub} - SP_{lb}$. Here SP_{ub} denotes an upper bound of SP, and SP_{lb} represents the lower bound of SP obtained by solving the LRP. To attain the optimal solution, our approach simply requires us to enumerate all columns whose reduced costs do not exceed $SP_{ub} - SP_{lb}$. Subsequently, we set these columns to $\gamma_l = \{0, 1\}$ to solve the original SP model. And the integer solution is an optimal solution.

To get an integer solution through linear solution, let us first introduce a Lemma.

Lemma 2. *For the relaxed SP (LRP), any linear solution can be converted into an integer solution. Furthermore, if the linear solution is optimal, then the resulting integer solution is also optimal for the origin SP.*

Proof. Proof of lemma 2. According to the definition of tandem sequence, a feasible schedule $l \in L_k$ is already a feasible solution at node k . And l contains all jobs and machines at node k . Assuming there exists an optimal solution $\{\gamma_{l_1}, \gamma_{l_2}, \dots, \gamma_{l_n}\}$ with fractional values in the LRP, that is

$$SP_{lp} = \min \sum_{l \in L_k} R_l \gamma_l = R_{l_1} \gamma_{l_1} + R_{l_2} \gamma_{l_2} + \dots + R_{l_n} \gamma_{l_n}$$

Based on constraints (53), we have $\gamma_{l_1} + \gamma_{l_2} + \dots + \gamma_{l_n} = 1$, $\gamma_{l_1}, \gamma_{l_2}, \dots, \gamma_{l_n} \in [0, 1]$. If $R_{l_s} = \min\{R_{l_1}, R_{l_2}, \dots, R_{l_n}\}$, its objective can be computed as $SP_{lp} = R_{l_1} \gamma_{l_1} + R_{l_2} \gamma_{l_2} + \dots + R_{l_n} \gamma_{l_n} \geq R_{l_s}(\gamma_{l_1} + \gamma_{l_2} + \dots + \gamma_{l_n}) = R_{l_s}$. Thus, we can construct a new solution by setting $\gamma_{l_s} = 1$ and $\gamma_{l_k} = 0$ for all $k \in \{1, \dots, n\} \setminus \{s\}$. This construction yields a solution with the lowest cost, which serves as the upper bound of the SP, i.e., $SP_{lp} \geq R_{l_s} = SP_{up}$, that is to say, $SP_{ub} = SP_{lb}$. Since the optimal solution of the linear relaxation is obtained by enumerating all columns with reduced costs no greater than $SP_{up} - SP_{lp} = 0$ and then solving the linear model, and at this time $SP_{lp} = SP_{up}$, according to Baldacci et al. (2008), we can conclude that this solution is the optimal one. $\square$

Based on Baldacci et al. (2008) and Lemma 2, we outline the complete procedure for solving the LRP by column generation algorithm.

Step 1: Initialize the set partitioning model and generate the initial schedule (tandem sequence) set L_k using Algorithm 1.

Step 2: Solve the LRP and obtain the optimal linear solution SP_{lb} .

Step 3: According to Lemma 2, we can convert SP_{tb} to an integer solution. This integer solution also is an optimal solution.

In summary, we can quickly obtain the optimal solution to the SP through the above steps.

6. Acceleration Strategies

It can be observed that, due to the presence of unused machines, when extending the label to a new job, an additional decision needs to be made regarding whether extend to an unused machine to form a new sequence. This significantly increases the number of expandable labels. Therefore, it becomes crucial to accelerate the algorithm. In the following subsections, three algorithmic acceleration strategies will be employed to improve the computational speed of the algorithm.

6.1. Dominance Rule

In terms of the updating functions in Section 5.3, all possible extensions will be performed and stored as labels. The algorithm in principle enumerates all feasible schedules by starting with the initial labels, which can be computationally expensive. Therefore, a dominance rule is proposed to remove the dominated labels during the extension. Let $\mathcal{L}_e$ denote the extension of label $\mathcal{L}$, and $\mathcal{L} \oplus \mathcal{L}_e$ denote a complete schedule. According to Dabia et al. (2013), label $\mathcal{L}^1$ dominates label $\mathcal{L}^2$ if the following two standard conditions hold:

- (i) All the complete schedule extensions that are feasible to label $\mathcal{L}^2$ can also be feasible to label $\mathcal{L}^1$, $\{\mathcal{L}_e^2\} \subseteq \{\mathcal{L}_e^1\}$.
- (ii) The reduced cost of $\mathcal{L}^1 \oplus \mathcal{L}_e^2$ is less than or equal to that of $\mathcal{L}^2 \oplus \mathcal{L}_e^2$, that is $c'(\mathcal{L}^1 \oplus \mathcal{L}_e^2) \leq c'(\mathcal{L}^2 \oplus \mathcal{L}_e^2)$.

Dominance 1.

$$c'(\mathcal{L}^1) \leq c'(\mathcal{L}^2) \quad (57)$$

$$r(\mathcal{L}^1) \leq r(\mathcal{L}^2) \quad (58)$$

$$\eta(\mathcal{L}^1) = \eta(\mathcal{L}^2) \quad (59)$$

$$\tau(\mathcal{L}^1) \leq \tau(\mathcal{L}^2) \quad (60)$$

$$\pi(\mathcal{L}^1) = \pi(\mathcal{L}^2) \quad (61)$$

$$N(\mathcal{L}^1) \supseteq N(\mathcal{L}^2) \quad (62)$$

$$\Delta(\mathcal{L}^1) \supseteq \Delta(\mathcal{L}^2) \quad (63)$$

In the context of label dominance, the reduced cost superiority of label $\mathcal{L}^1$ over label $\mathcal{L}^2$ is constrained by equation (57). For two labels with the same currently processed job, equations (58) - (63) jointly restrict the feasible extension sequence set of label $\mathcal{L}^2$ to be a subset of the feasible extension sequence set of label $\mathcal{L}^1$. This ensures that any feasible extension of label $\mathcal{L}^2$ is also a feasible extension of label $\mathcal{L}^1$. The proof of Dominance 1 can be found in [Appendix D](#).

6.2. An Upper Bound of UPMSPST

In label setting algorithm, the makespan time of each machine plays a crucial role in the extension process. A longer makespan increases the complexity and cost of generating labels in the solution space. However, if we can identify an effective upper bound for the makespan, it will significantly reduce the search space of the algorithm. Let $\xi_h = \max_{g \in C_k, m \in M} \{e_{g,h} + p_{h,m}\}$ be the estimation of the sum of processing time and setup time for job h and $\xi_h^* = \max_{h \in C_k} \{\xi_h\}$, where m represents the machine number. Based on this, we propose an upper bound $\bar{C}_{max}$ of makespan and lemma 3:

$$\bar{C}_{max} \leq \frac{(\sum_{h \in C_k} \xi_h) - \xi_h^*}{m} + \xi_h^* \quad (64)$$

Let $\{E_1, E_2, \dots, E_m\}$ denote the completion time of each machine, and let $E_m = E^* = \bar{C}_{max}$ represents the optimal makespan (maximum of completion time). We have

Lemma 3. $\forall E_a, E_b \in \{E_1, \dots, E^*, \dots, E_m\}, \quad E_a - E_b \leq \xi_h^* \quad \text{holds.}$

Proof. Proof of lemma 3. Assuming the existence of maximum completion time E^* and minimum completion time E_a , such that

$$E^* - E_a > \xi_h^*$$

If any job h processed on machine E^* is taken and assigned to machine E_a for processing, then:

$$\begin{aligned} E^* &> E^* - \xi_h \\ E_a &< E_a + \xi_h \leq E_a + \xi_h^* < E^* \end{aligned}$$

According to the definition of makespan, E^* is not the optimal makespan of the completion time, which is contrary to it, the hypothesis is not valid, and the original reasoning is supported. $\square$

Proof. Proof of UB (64). According to lemma 3:

$$\begin{aligned} \bar{C}_{max} - E_1 &\leq \xi_h^* \\ \Leftrightarrow \bar{C}_{max} - E_1 + \dots + \bar{C}_{max} - E_{m-1} &\leq \xi_h^* + \dots \xi_h^* \\ \Leftrightarrow (m-1)\bar{C}_{max} - (E_1 + \dots + E_{m-1}) &\leq (m-1)\xi_h^* \\ \Leftrightarrow (m-1)\bar{C}_{max} - (E_1 + \dots + E_{m-1}) + \bar{C}_{max} - E_m &\leq (m-1)\xi_h^* + 0 \\ \Leftrightarrow m\bar{C}_{max} - (E_1 + \dots + E_{m-1} + E_m) &\leq (m-1)\xi_h^* \\ \Leftrightarrow m\bar{C}_{max} - \sum_{h \in C_k} \xi_h &\leq (m-1)\xi_h^* \\ \Leftrightarrow \bar{C}_{max} &\leq \frac{\sum_{h \in C_k} \xi_h}{m} + \frac{(m-1)\xi_h^*}{m} \\ \Leftrightarrow \bar{C}_{max} &\leq \frac{(\sum_{h \in C_k} \xi_h) - \xi_h^*}{m} + \frac{\xi_h^*}{m} + \frac{(m-1)\xi_h^*}{m} \\ \Leftrightarrow \bar{C}_{max} &\leq \frac{(\sum_{h \in C_k} \xi_h) - \xi_h^*}{m} + \xi_h^* \end{aligned}$$

$\square$

In summary, this section presents an effective upper bound for SPs, which can serve as a cutoff time to reduce the search space during label extension. In theory, this can expedite the algorithm's search speed. The detailed acceleration effect will be elucidated in the experimental section.

6.3. Deduplication of Tandem Sequence

Due to the inherent characteristics of tandem sequences, different schedules can yield the same solution. As depicted in Figure 11, Schedule 1 has a machine sequence of (3, 2, 1), while Schedule 2 has a machine sequence of (2, 1, 3). However, the machines in these sequences operate in parallel to process the jobs, without any specific order or precedence. Therefore, both Schedule 1 and Schedule 2 yield the same result (See the right side of Figure 11).

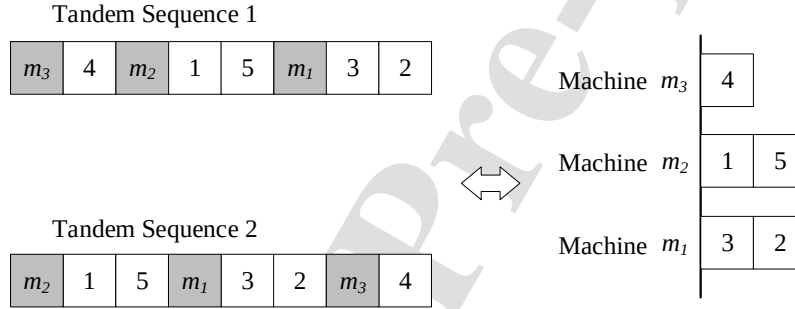


Figure 11: An example of two different schedules with same solution.

Therefore, for different tandem sequences, we normalize them as shown in Figure 12. In the normalized sequence, the machine sequence is arranged in ascending order by default. After normalizing the different tandem sequences, we retain the unique sequences and remove the duplicate ones, completing the deduplication process.

7. Computational results

Then, the effectiveness of the LBB algorithm designed in this paper and the tandem-sequence-based column generation will be tested. As M3DPSPST

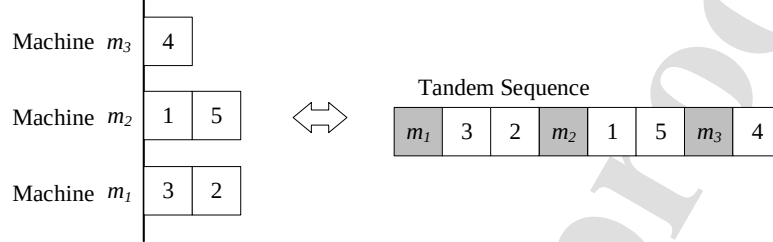


Figure 12: Normalization of tandem sequence.

is a novel combinatorial optimization problem, there are currently no existing benchmark instances available in the literature. However, due to the specific nature of the two-stage problem, the optimization problems for each stage can be separately generated based on existing benchmark instances. Therefore, this paper generates standard test instances for evaluation based on benchmark instances related to [Missaoui and Ruiz \(2022\)](#) and [Calmels \(2022\)](#). Following [Missaoui and Ruiz \(2022\)](#), we do not employ parallel coding or computing. We also specify the values for the number of nodes, the number of machines, and the number of jobs, and increase the number of replicates included in each test case to 10. According to [Calmels \(2022\)](#), we generate the processing time and setup time using a uniform distribution, and randomly generate the number of jobs at each node. Additionally, we extend the runtime of each test case from 3600 seconds to 7200 seconds. All models and methods in this paper were implemented using Java and executed on a workstation equipped with an Intel(R) Core(TM) i5-8400 2.80GHz processor and 16GB of memory, running on the Windows 10 operating system. The runtime limit for each test case is set to 7200 seconds. The BMP solver used was ILOG CPLEX 20.1 . The instances data and necessary results can be found on GitHub (<https://github.com/lfc38461/M3DPSPST>).

7.1. Instance sets

A total of three sets of instances were generated and tested in this section, labeled as C2, C3, and C4, corresponding to the number of customer nodes $n \in \{2, 3, 4\}$. The C2 instance set consists of 8 groups, representing different combinations of $m \in \{2, 3, 4\}$ processing machines and $g \in \{5, 8, 10\}$ jobs, with each group having $n = 2$ customer nodes. Each group of instances

contains 10 test cases, resulting in a total of 80 test cases in the C2 instance set. The processing times of jobs on different machines follow a uniform distribution between 1 and 15, while the setup times between jobs follow a uniform distribution between 0 and 20.

The C3 instance set consists of 12 groups, representing different combinations of $m \in \{2, 3, 4\}$ processing machines and $g \in \{5, 8, 10, 15\}$ jobs, with each group having $n = 3$ customer nodes. Each group of instances contains 10 test cases, resulting in a total of 120 test cases in the C3 instance set. The processing times of jobs on different machines follow a uniform distribution between 1 and 15, while the setup times between jobs follow a uniform distribution between 0 and 20.

The C4 instance set consists of 12 groups, representing different combinations of $m \in \{2, 3, 4\}$ processing machines and $g \in \{5, 8, 10, 15\}$ jobs, with each group having $n = 4$ customer nodes. Each group of instances contains 10 test cases, resulting in a total of 120 test cases in the C4 instance set. The processing times of jobs on different machines follow a uniform distribution between 1 and 15, while the setup times between jobs follow a uniform distribution between 0 and 20.

In the experimental result tables, MILP represents the result obtained by directly solving the MILP model. LBBD represents the result obtained using the proposed algorithm based on the tandem-sequence-based column generation and logic-based Benders decomposition. GAP_{LB} (%) and GAP_{UB} (%) indicate the lower bound gap and upper bound gap of the current instance solved by the algorithm. The specific calculation formulas are as follows: $GAP_{LB} = \frac{UB^* - LB'}{UB^*} \times 100\%$ and $GAP_{UB} = \frac{UB' - UB^*}{UB^*} \times 100\%$, where LB' and UB' represent the lower bound and upper bound obtained by the current algorithm, and UB^* represents the best upper bound obtained by MILP and LBBD. Time (s) indicates the average runtime of the algorithm in seconds. Each case is an average of 10 test instances.

7.2. Comparisons between MILP and LBBD on Instances C2, C3

Due to the limitations of the MILP solver's solving capability, this section compares the performance of the MILP solver and the LBBD algorithm on the C2 and C3 instance sets. As shown in Table 3, in the 8 groups of instances in the C2 instance set, both MILP and LBBD are able to find the optimal solution within the given time limit, and the upper and lower bounds obtained by both methods eventually converge. In terms of average runtime, LBBD performs slightly better. In relatively simple instances, such

Table 3: Experimental Results of MILP and LBBD on Instance C2

n	m	g	MILP			LBBD		
			GAP_{LB}	GAP_{UB}	Time	GAP_{LB}	GAP_{UB}	Time
2	2	5	0.00	0.00	0.11	0.00	0.00	0.06
2	2	8	0.00	0.00	15.91	0.00	0.00	4.71
2	2	10	0.00	0.00	1231.33	0.00	0.00	255.08
2	3	5	0.00	0.00	0.15	0.00	0.00	0.11
2	3	8	0.00	0.00	29.46	0.00	0.00	18.97
2	3	10	0.00	0.00	1795.15	0.00	0.00	150.54
2	4	5	0.00	0.00	1.02	0.00	0.00	0.79
2	4	8	0.00	0.00	34.36	0.00	0.00	84.58
Average			0.00	0.00	388.44	0.00	0.00	64.36

as $\{n, m, g\} \in \{\{2, 2, 5\}, \{2, 2, 8\}, \{2, 3, 5\}, \{2, 3, 8\}, \{2, 4, 5\}\}$, the runtime of MILP is comparable to that of LBBD. However, for other instances, the solving time of MILP is significantly longer than that of LBBD. In some cases, the average runtime of MILP approaches 1500 seconds, while the average solving time of LBBD is around 250 seconds. Overall, the average runtimes of the two methods on small-scale instances are 388.44 seconds and 64.36 seconds, respectively, with MILP taking more than sixth as long as LBBD on average to solve instances.

In the C3 instance set (Table 4), “OFM” (Out of Memory) indicates the number of instances, out of ten, that could not be solved due to memory overflow. It is worth noting that for the C3 instance set with $g \in \{5, 8, 10\}$, both MILP and LBBD converge to the optimal solution within 7200 seconds. However, for instances with $g = 15$, MILP encounters relative difficulty, resulting in 13 instances that could not obtain results due to OFM errors. While the remaining instances can be solved, the upper and lower bounds do not fully converge. On average, there is a 70% gap in the lower bounds, and a 40% gap in the upper bounds, indicating that many instances in the remaining set cannot converge to the optimal solution within 7200 seconds.

As for the LBBD algorithm, only the instance $\{3, 4, 15\}$ fails to converge to the optimal solution, while the rest of the instances can be solved in a relatively short time. On average, MILP has an average upper bound

gap of 10.18%, significantly higher than LBBD's average GAP_{UB} of 0.27%. The average runtime of MILP is 1872.06 seconds, while LBBD takes 692.63 seconds. MILP encounters an average of 1.08 instances where OFM errors prevent solving, while LBBD does not encounter any instances with OFM errors.

Table 4: Experimental Results of MILP and LBBD on Instance C3

n	m	g	MILP				LBBD		
			GAP_{LB}	GAP_{UB}	Time	OFM	GAP_{LB}	GAP_{UB}	Time
3	2	5	0.00	0.00	0.16	0	0.00	0.00	0.54
3	2	8	0.00	0.00	2.12	0	0.00	0.00	1.24
3	2	10	0.00	0.00	47.29	0	0.00	0.00	0.93
3	2	15	71.03	27.30	7200.00	3	0.00	0.00	51.03
3	3	5	0.00	0.00	0.32	0	0.00	0.00	3.28
3	3	8	0.00	0.00	4.38	0	0.00	0.00	4.23
3	3	10	0.00	0.00	171.43	0	0.00	0.00	8.67
3	3	15	77.18	46.83	7200.00	5	0.00	0.00	779.13
3	4	5	0.00	0.00	0.58	0	0.00	0.00	30.41
3	4	8	0.00	0.00	13.53	0	0.00	0.00	58.21
3	4	10	0.00	0.00	624.94	0	0.00	0.00	173.83
3	4	15	82.44	48.04	7200.00	5	42.91	3.21	7200.00
Average			19.22	10.18	1872.06	1.08	3.58	0.27	692.63

In conclusion, although MILP may outperform LBBD in terms of solving efficiency in some individual instances, overall average performance shows that LBBD performs better in terms of solution accuracy, solving time, and memory usage.

7.3. Experimental results of LBBD on Instance C4

Due to the limitations of the MILP solver's solving capability, this subsection presents the experimental results regarding the performance of the LBBD algorithm on C4 instance set. In Table 5, instances $\{\{4, 2, 5\}, \{4, 2, 8\}, \{4, 2, 10\}, \{4, 2, 15\}\}$ all converge to the optimal solution within the specified time. And the average runtimes below 100 seconds. For instances $\{\{4, 3, 5\}, \{4, 3, 8\}, \{4, 3, 10\}, \{4, 3, 15\}\}$, the LBBD algorithm is able to achieve the

Table 5: Experimental Results of LBBD on Instance C4

n	m	g	LBBD		
			GAP_{LB}	GAP_{UB}	Time
4	2	5	0.00	0.00	54.04
4	2	8	0.00	0.00	73.36
4	2	10	0.00	0.00	73.51
4	2	15	0.00	0.00	87.41
4	3	5	2.26	0.00	4041.65
4	3	8	0.92	0.00	4750.96
4	3	10	1.77	0.00	5253.86
4	3	15	8.25	0.00	6646.46
4	4	5	34.26	0.00	7200.00
4	4	8	36.54	0.00	7200.00
4	4	10	37.75	0.00	7200.00
4	4	15	38.42	0.00	7200.00
Average			13.35	0.00	4148.44

optimal solution for some instances, but not all instances converge within 7200 seconds. The average lower bound gaps obtained for these instances are below 10%.

For other instances in the C4 instance set, the LBBD algorithm fails to obtain the optimal solution within the specified time. It can be observed that as the instance size increases, the gap in lower bounds also increases. The average lower bound achieved by LBBD on large-scale instances is 13.35%, with an average runtime of 4148.44 seconds.

7.4. Impact of Acceleration Strategies

To validate the effectiveness of the three acceleration strategies, this section tests their impacts on the C2, C3, and C4 instance sets individually. The results are presented in detail in Tables 6, 7, and 8.

Table 6: Impact of acceleration strategies on LBBD in C2

n	m	g	TL	LET	SL	DL	ITER	Ltime	Ctime
2	2	5	2152.60	371.50	66.70	1993.70	4.00	0.01	0.05
2	2	8	70994.50	7043.90	7267.40	108204.70	4.00	0.01	4.69
2	2	10	460768.10	38122.80	74995.80	960378.30	4.00	0.01	255.06
2	3	5	11295.20	2209.50	458.20	12385.10	8.00	0.04	0.07
2	3	8	194383.70	36334.40	22441.00	400223.80	8.00	0.08	18.90
2	3	10	646489.60	117415.40	105752.20	1864072.70	8.00	0.05	150.49
2	4	5	58571.10	10482.50	3457.00	71987.00	16.00	0.14	0.65
2	4	8	680931.70	171531.80	73286.60	1620504.30	16.00	0.11	84.47
Average			282828.89	50635.79	39948.06	660787.15	8.50	0.06	64.30

In these tables, “TL” (Total Labels) represents the total number of labels generated by the column generation algorithm based on the tandem sequence. “LET” (Labels Eliminated by T) represents the sum of labels eliminated due to their maximum completion time exceeding the effective upper bound. “SL” (Same Labels) represents the sum of duplicate labels after deduplication. “DL” (Dominated Labels) represents the number of labels eliminated by the dominance rule. “ITER” indicates the number of iterations performed by the algorithm. To compare the runtime of different components of the algorithm, “Ltime” (LBBD time) is used to represent the time spent by the LBBD algorithm outside of solving the SPs (primarily the

Table 7: Impact of acceleration strategies on LBBD in C3

n	m	g	TL	LET	SL	DL	ITER	Ltime	Ctime
3	2	5	3787.90	538.40	55.00	1837.40	26.20	0.37	0.18
3	2	8	12423.70	1864.00	334.60	9320.10	28.40	0.95	0.29
3	2	10	38611.50	6882.40	1873.00	41670.60	27.00	0.55	0.38
3	2	15	589147.30	59715.70	98160.80	1135193.80	30.60	0.56	50.47
3	3	5	47746.00	8741.80	972.90	31647.40	126.60	2.20	1.08
3	3	8	140161.00	27454.50	4652.90	130616.00	135.30	2.62	1.61
3	3	10	448417.10	98917.90	26695.00	552294.60	139.80	2.73	5.94
3	3	15	5384924.80	1150509.90	868833.50	12015459.10	141.90	2.79	776.34
3	4	5	529902.60	98105.20	23206.90	393084.70	509.80	24.12	6.30
3	4	8	1718956.80	346243.60	96046.20	1857184.00	588.60	34.96	23.25
3	4	10	4801400.90	1144368.40	345085.60	6740736.70	588.60	35.13	138.70
3	4	15	19801290.80	4648523.70	2745442.80	49301742.30	268.40	6.49	7193.51
Average			2793064.20	632655.46	350946.60	6017565.56	217.60	9.46	683.17

time taken to solve the MP), while “Ctime” (Column generation time) denotes the time spent on solving the SPs using the column generation method. We have $\text{Time} = \text{Ltime} + \text{Ctime}$.

Although the number of labels generated by the C2 instance set is relatively small, the maximum number of labels has already reached the million level. To clearly demonstrate the proportion of different types of labels generated during the solving process for each group of instances in the C2 instance set, we calculated the proportions using the formulas $\frac{TL}{TL+LET+SL+DL} \times 100\%$, $\frac{LET}{TL+LET+SL+DL} \times 100\%$, $\frac{SL}{TL+LET+SL+DL} \times 100\%$, $\frac{DL}{TL+LET+SL+DL} \times 100\%$ and presented them in Figure 13. The x-axis represents the proportion, while the y-axis represents the different groups of instances in the C2 instance set, labeled as C2-1 to C2-8.

Among these, the average proportion of “DL” exceeds 50%, indicating that the dominance rule effectively eliminates a significant portion of irrelevant labels. The proportions of “LET” and “SL” are relatively small. It may vary in individual instances. Although the number of duplicate labels and labels exceeding the effective upper bound is small, the acceleration strategies effectively identify and eliminate these labels to expedite the solving process.

Similarly, in the C3 instance set, the average total number of labels

Table 8: Impact of acceleration strategies on LBBD in C4

n	m	g	TL	LET	SL	DL	ITER	Ltime	Ctime
4	2	5	19322.80	1439.40	0.00	3601.10	340.20	50.77	3.27
4	2	8	144939.70	15759.50	3106.50	117393.10	429.00	68.72	4.64
4	2	10	163878.30	18920.90	4510.00	115569.20	417.40	68.77	4.74
4	2	15	607670.20	67570.00	24385.00	621662.40	471.00	80.55	6.86
4	3	5	494585.80	61276.80	7539.50	144805.80	3573.30	3998.65	43.00
4	3	8	3383229.90	627858.40	108598.80	3356711.30	3992.90	4695.35	55.61
4	3	10	3615863.30	643287.40	105431.90	3325531.50	4133.30	5193.42	60.44
4	3	15	13765694.30	2667182.70	678490.00	16911545.60	4503.10	6515.44	131.02
4	4	5	3014348.20	398940.00	137441.80	1205106.40	6403.90	7109.18	90.82
4	4	8	16941924.30	3184191.20	836833.90	19135792.60	6457.90	6988.99	211.01
4	4	10	19894427.60	3411090.70	960728.90	21332904.90	6295.00	6956.83	243.17
4	4	15	55639038.60	13060372.60	3346448.30	77745634.50	6058.10	6215.20	984.80
Average			9807076.92	2013157.47	517792.88	12001354.87	3589.59	3995.16	153.28

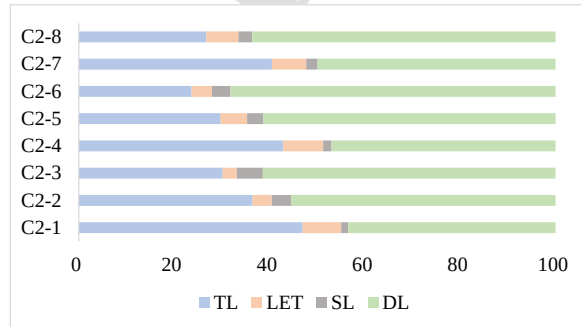


Figure 13: Proportion of label types in C2

reaches 2793064.20, and the number of labels eliminated by the dominance rule is more than twice as much as TL. As the instance size increases, the number of labels eliminated by the dominance rule shows an upward trend. Therefore, the proposed dominance rule is effective and necessary. The number of the other two types of labels are 632655.46 and 350946.60, respectively. Additionally, the number of iterations increases with the instance size, with some instances requiring nearly 600 iterations. Figure 14 provides a detailed visualization of the proportions of different types of labels for each group of instances (C3-1 to C3-12).

In the C4 instance set, as the instance size increases, the problem becomes more complex, and the number of various labels also show an increasing trend. The total number of labels for the $\{4, 3, 15\}$ instance exceeds tens of millions, which leads to the inability to solve the instance within the specified time. Consequently, the proportion of labels eliminated by the dominance rule decreases in certain instances. The number of iterations also increases, with some instances requiring nearly 7000 iterations. Figure 15 provides a detailed visualization of the proportions of different types of labels for each group of instances (C4-1 to C4-12).

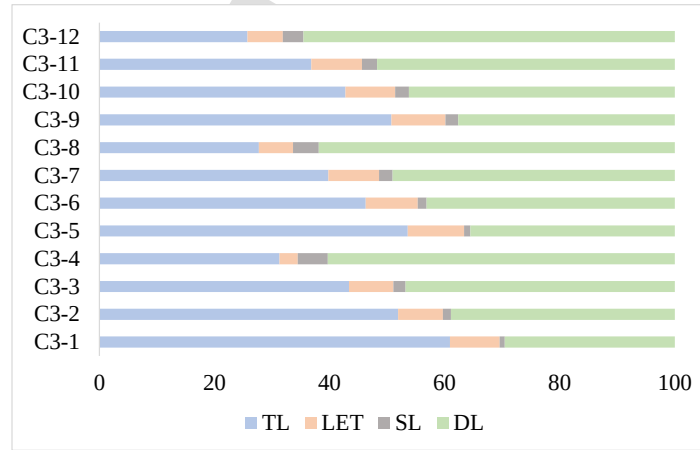


Figure 14: Proportion of label types in C3

Based on the runtime analysis in Tables 6, 7, and 8, the bottlenecks affecting the algorithm's performance vary with the problem size. In C2, where the problem size is relatively small, the time spent solving the master problem in LBBB is negligible. At this stage, the column generation for SP resolution is the primary bottleneck. For example, in instance $\{2,2,10\}$, "Ctime" takes nearly 260 seconds, while "Ltime" requires only 0.01 seconds. In C3, although "Ctime" still accounts for most of the runtime, the gap between "Ctime" and "Ltime" decreases. Notably, in instances $\{3,4,5\}$ and $\{3,4,8\}$, "Ltime" exceeds "Ctime". In C4, as the problem size increases, "Ltime" becomes the primary bottleneck. This is mainly because "Ltime" involves solving the master problem directly with CPLEX, while "Ctime" benefits from a well-designed column generation algorithm for SPs. As the problem size grows, CPLEX's inability to leverage problem-specific acceleration techniques increasingly turns "Ltime" into the runtime bottleneck.

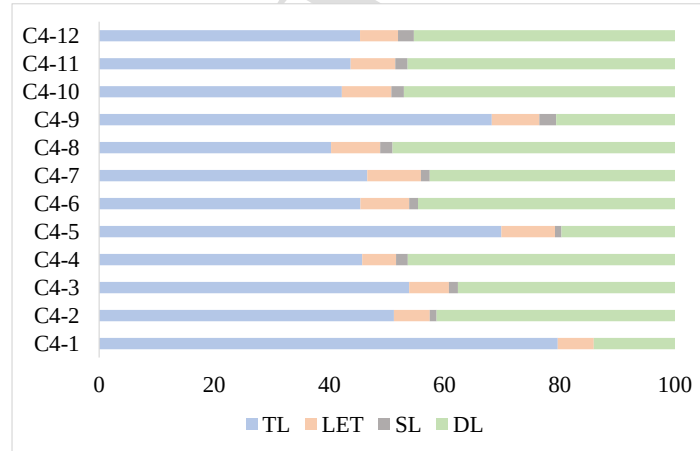


Figure 15: Proportion of label types in C4

7.5. A Practical Instance Set

To evaluate the performance of the MILP model and the LBBD algorithm on real-world instances, a small dataset of actual cases, labeled as Set A, is tested. This set includes 14 instances (A01-A14) where customer nodes and depots are located within $114-115^\circ$ longitude and $30-31^\circ$ latitude in Wuhan, Hubei Province. Each instance uses a 3D printer rental or manufacturing company, such as Wuhan Doit Technology Co., Ltd., as the depot, and customer nodes are large hospitals, research institutes, and smart manufacturing companies in Wuhan, such as Wuhan First Hospital (see Figure 16). The instances in Set A have parameters of $n \in \{3, 4, 5\}$, $m \in \{2, 3\}$, and $g \in \{5, 10, 15, 20\}$. The speed of the vehicle is 36 km/h. The completion time for each 3D printing task ranges from 10 to 30 minutes, with a setup time between tasks of 1 to 10 minutes.

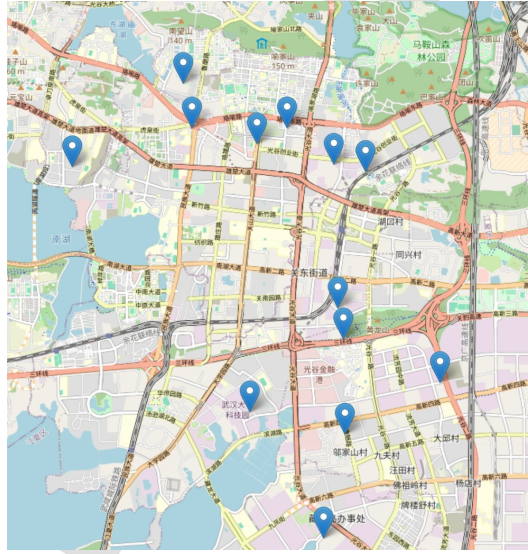


Figure 16: Locations of some customer nodes in Set A.

The results in Table 9 indicate that both MILP and LBBD solve the first eight instances (A01-A08) to optimality. While MILP shows slightly better performance of runtime on A03 and A04, LBBD significantly outperform

MILP in solving the other instances. For instances A09-A14, MILP fails to find the optimal solution within the 7200 second limit, whereas LBBD is able to solve all but A14 to optimality within 2000 seconds. On average, LBBD demonstrates superior solution time and quality compared to MILP.

We also collect data on the number of iterations, run time, and various label statistics for the LBBD algorithm across the instances in set A, as detailed in Table 10. The results show that “TL” and “DL” labels constitute a significant proportion of the total labels, with varying distributions across different instances. For example, “TL” labels outnumber “DL” labels in instances A01-A08, while the opposite is true for instances A09-A12. Labels “LET” and “SL” represent a smaller share in all instances. Additionally, the runtime analysis reveals that for instances A01-A08, the “Ltime” and “Ctime” are similar due to the smaller instance sizes, allowing for quick solutions. However, as instance sizes increase in A09-A12, “Ctime” significantly exceeds “Ltime”, indicating that the column generation for solving SP primarily impacts solution speed. In contrast, for instances A13 and A14, “Ltime” accounts for a larger portion of the total run time, suggesting that solving the master problem in LBBD consumes considerable time.

7.6. Sensitivity Analysis

In this subsection, we perform a sensitivity analysis to evaluate the influence of parameters n , m , and g on the optimal solutions obtained by the LBBD. Furthermore, we analyze the relationship between the maximum number of machines that can be allocated to each customer node and the actual number of machines allocated. To further examine the load distribution among machines, we discuss the number of jobs assigned to each machine. For this analysis, we select the following seven instances from C2-C4: $\{2,3,10\}$, $\{3,3,5\}$, $\{3,3,8\}$, $\{3,3,10\}$, $\{3,2,10\}$, $\{3,4,10\}$, and $\{4,3,10\}$, denoted as S1-S7.

Based on $\{S1, S4, S7\}$, the impact of the number of nodes n on runtime can be seen in Figure 17. As n increases from 2 to 3, the runtime decreases slightly. However, when n reaches 4, the runtime sharply increases to 6000 seconds. This indicates that the number of nodes significantly impacts the complexity of problem. In Figure 18, the results for instances $\{S5, S4, S6\}$ show that as the number of machines m increases, the solution speed decreases, with a noticeable spike in runtime when $m = 4$, though not as significant as the increase with n . Lastly, Figure 19 illustrates the relationship between the number of jobs g and runtime. We observe that as g increases from 5 to 8, the runtime actually decreases, while a slight increase is noted

Table 9: Experimental Results of MILP and LBBD on Set A

Index	n	m	g	MILP			LBBD		
				GAP_{LB}	GAP_{UB}	Time	GAP_{LB}	GAP_{UB}	Time
A01	3	2	10	0.00	0.00	36.08	0.00	0.00	0.63
A02	3	2	10	0.00	0.00	41.53	0.00	0.00	0.92
A03	3	2	5	0.00	0.00	0.13	0.00	0.00	0.31
A04	3	3	5	0.00	0.00	0.27	0.00	0.00	1.88
A05	3	3	10	0.00	0.00	252.89	0.00	0.00	5.30
A06	3	3	10	0.00	0.00	179.84	0.00	0.00	3.64
A07	3	3	10	0.00	0.00	175.32	0.00	0.00	2.61
A08	3	3	10	0.00	0.00	96.03	0.00	0.00	2.79
A09	3	2	15	49.27	0.00	7200.00	0.00	0.00	39.41
A10	3	3	15	62.65	0.00	7200.00	0.00	0.00	1423.41
A11	3	2	15	64.01	2.54	7200.00	0.00	0.00	35.15
A12	3	3	15	37.01	0.00	7200.00	0.00	0.00	1389.53
A13	4	2	15	50.22	0.12	7200.00	0.00	0.00	3.47
A14	5	3	20	47.97	0.00	7200.00	48.24	8.44	7200.00
Average				22.22	0.19	3141.58	3.45	0.60	722.07

Table 10: Impact of acceleration strategies on LBBD in Set A

Index	n	m	g	TL	LET	SL	DL	ITER	Ltime	Ctime
A01	3	2	10	14916	1390	408	9420	10	0.31	0.32
A02	3	2	10	18188	1108	626	10336	10	0.71	0.21
A03	3	2	5	2212	216	44	914	10	0.20	0.10
A04	3	3	5	29916	3544	708	13860	50	1.18	0.70
A05	3	3	10	213742	15096	8127	149132	50	3.21	2.08
A06	3	3	10	183144	17877	8499	135784	50	1.95	1.69
A07	3	3	10	174766	18379	5560	134117	50	1.09	1.52
A08	3	3	10	185940	18596	7366	138549	50	1.06	1.73
A09	3	2	15	352498	22741	40024	479792	10	0.10	39.31
A10	3	3	15	4614885	321844	491675	6696880	50	0.87	1422.54
A11	3	2	15	352552	21874	39810	483221	10	0.10	35.04
A12	3	3	15	4614885	321844	491675	6696880	50	0.83	1388.70
A13	4	2	15	61124	6596	2282	40418	28	2.97	0.49
A14	5	3	20	27208764	3421790	1532696	23685835	2402	7060.31	139.69
Average				2716252.29	299492.50	187821.43	2762509.86	202.14	505.35	216.72

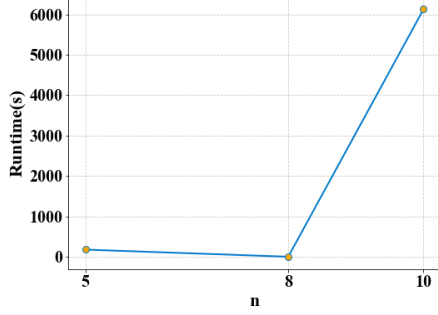


Figure 17: The impact of the number of nodes n on run time

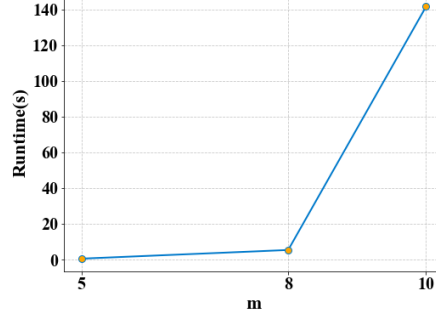


Figure 18: The impact of the number of machines m on run time

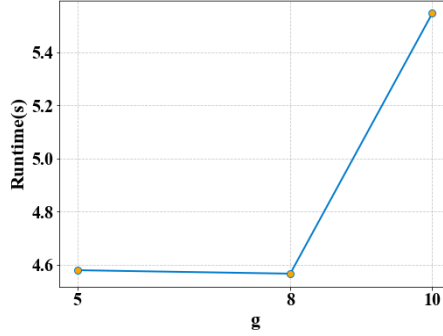


Figure 19: The impact of the number of jobs g on run time

when g rises from 8 to 10. Although we cannot definitively conclude the relationship between g and runtime, it does not imply that there is no connection between these two factors.

The relationship between the maximum number of machines (m) that can be allocated to a customer node and the actual average number of machine usage is depicted in Figure 20. The average number of machine usage (ANMU) is calculated by summing the actual machine allocations across all customer nodes and then averaging them. The machine utilization rate, denoted by R_m , can also be derived from this data ($R_m = \frac{ANMU}{m}$). As shown in the figure, in instances S1 and S5, the machine utilization rate reaches 100%,



Figure 20: The machine utilization rate.

indicating that all machines are allocated to each customer node during the vehicle's visit. In instance S7, the machine utilization rate is relatively high at 66%. In the remaining instances, the utilization rate is consistently at 50%, suggesting that, in most cases, allocating only a portion of the machines to each customer node and distributing the rest to other nodes yields a **near-optimal solution**. These findings highlight that allocating all machines to each customer node is not always the most efficient strategy.

Additionally, the balance in the number of jobs processed by each machine in the optimal solution is also worth discussing. As illustrated in Figure 21, max represents the maximum number of jobs processed by the machine, min denotes the minimum number of jobs processed by the machine, and $diff$ ($diff = max - min$) indicates the range of machine processing numbers. The chart clearly shows that in S5, where $diff$ equals 0, all machines process the same number of jobs. The largest difference occurs in S3, where the machine with the most processing number handles 4 more jobs than the one with the least. The other instances exhibit a more balanced range, with differences ranging from 1 to 2. Based on these findings, it can be concluded that in the optimal solution, the distribution of jobs across machines tends to be relatively balanced, with large disparities being uncommon.

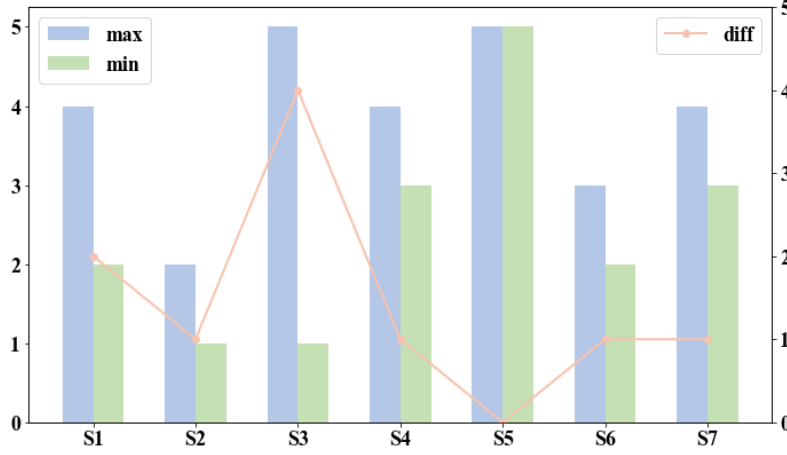


Figure 21: The range of machine processing numbers.

7.7. Exploration of Solutions for Large Scale Instances

In this sub-section, we generate a larger scale instance set, CL instances, to test the performance of our algorithm. The detailed experimental results can be found in [Appendix A](#). Meanwhile, we developed a heuristic algorithm (HLBBD) based on the LBBD algorithm presented in this paper. By comparing the performances of HLBBD and LBBD across all instance sets, we found that HLBBD not only achieves high-quality solutions in a short time when solving large scale instances but also obtains the optimal solution for some small scale instances. For detailed test results, please refer to [Appendix B](#) and [Appendix C](#).

8. Conclusion

This study addresses a novel Mobile 3D Printer Scheduling Problem. The problem involves a truck loaded with machines departing from a distribution center, visiting a series of customer nodes, and delivering the machines for processing. Once the machines complete their processing tasks, the truck returns to the satellite nodes to retrieve the machines and transport them back to the distribution center.

To provide a detailed mathematical description of the problem, an MILP model is proposed. Considering that M3DPSPST involves both transporta-

tion and production stages, we present a logic-based Benders decomposition algorithm. The master problem solves the pickup and delivery problem for truck transportation, while the SP addresses the parallel machine scheduling problem at each customer node. Based on the characteristics of the SP, a tandem-sequence-based column generation is designed, and acceleration strategies such as sequence deduplication, dominance rules, and effective upper bounds are employed to enhance the algorithm's efficiency.

Finally, 3 sets of generated instances and a set of real-world instances are established to experimentally test the algorithm. The efficiency of the proposed algorithm framework is demonstrated through a comparison between the LBB algorithm and the MILP model. Furthermore, a comparison is made regarding the quantities of various types of labels generated during the column generation process, thereby providing evidence of the impact of the three acceleration strategies.

Based on the experimental results and sensitivity analysis, this study provides the following management insights for businesses:

- For enterprises, MILP can partially meet their solving needs. However, as the scale of instances increases, the problem complexity rises, and employing precise algorithms based on problem structure becomes a more effective approach.
- When allocating machines to each customer node, adopting a strategy of allocating all machines may not always be optimal. It is necessary to consider the task volume and processing speed at each customer node comprehensively.
- It is essential to balance the workload among machines, especially to prevent scenarios where some machines are overloaded while others are underutilized. Proper allocation of processing resources can significantly enhance production efficiency.
- The mobile machine production model represents a new processing method in the sharing economy. While it can enhance economic efficiency and optimize the allocation of social resources for enterprises, it also imposes higher requirements on overall supply chain optimization.

The future directions can be outlined as follows: First, the TSP with delivery and pick-up problem in the first stage can be extended to a VRP with delivery

and pick-up, presenting an extreme challenge. Second, constraints such as time windows and release times can be considered in this problem. Third, the tandem sequence-based algorithm is particularly effective for small to medium scale instances; however, designing a heuristic or an effective exact method for large-scale instances can be very valuable.

References

- 3ders, 2018. Amazon awarded patent for innovative new on-demand 3d printing retail service. URL: <https://www.3ders.org/>. accessed: 2024-07-02.
- Alarcon-Gerbier, E., Zipfel, B., Buscher, U., 2022. Customer order scheduling in a mobile 3d printing factory. *IFAC-PapersOnLine* 55, 2270–2275.
- Alfieri, A., Druetto, A., Grosso, A., Salassa, F., 2021. Column generation for minimizing total completion time in a parallel-batching environment. *Journal of Scheduling* 24, 569–588.
- Alvelos, F., Lopes, M., Lopes, H., 2016. A matheuristic based on column generation for parallel machine scheduling with sequence dependent setup times, in: *Computational Management Science: State of the Art 2014*, Springer. pp. 233–238.
- Baldacci, R., Christofides, N., Mingozzi, A., 2008. An exact algorithm for the vehicle routing problem based on the set partitioning formulation with additional cuts. *Mathematical Programming* 115, 351–385.
- Benjaafar, S., Hu, M., 2020. Operations management in the age of the sharing economy: What is old and what is new? *Manufacturing & Service Operations Management* 22, 93–101.
- Breugem, T., Van Rossum, B., Dollevoet, T., Huisman, D., 2022. A column generation approach for the integrated crew re-planning problem. *Omega* 107, 102555.
- Cai, L., Li, J., Wang, K., Luo, Z., Qin, H., 2025. Optimal allocation and route design for station-based drone inspection of large-scale facilities. *Omega* 130, 103172. URL: <https://www.sciencedirect.com/science/article/pii/S0305048324001373>, doi:<https://doi.org/10.1016/j.omega.2024.103172>.

- Calmels, D., 2022. An iterated local search procedure for the job sequencing and tool switching problem with non-identical parallel machines. *European Journal of Operational Research* 297, 66–85.
- CassaMobile, 2013. Flexible mini-factory for local and customized production in a container. URL: <https://www.cassamobile.eu/>. accessed: 2024-07-02.
- Chen, Z.L., Powell, W.B., 1999. Solving parallel machine scheduling problems by column generation. *INFORMS Journal on Computing* 11, 78–94.
- Dabia, S., Ropke, S., van Woensel, T., De Kok, T., 2013. Branch and price for the time-dependent vehicle routing problem with time windows. *Transportation Science* 47, 380–396. URL: <https://doi.org/10.1287/trsc.1120.0445>.
- Didi, 2024. More than a journey. URL: <https://www.didiglobal.com/>. accessed: 2024-07-02.
- Druetto, A., Pastore, E., Renner, E., 2023. Parallel batching with multi-size jobs and incompatible job families. *Top* 31, 440–458.
- Güden, H., Süral, H., 2014. Locating mobile facilities in railway construction management. *Omega* 45, 71–79.
- Halper, R., Raghavan, S., 2011. The mobile facility routing problem. *Transportation Science* 45, 413–434.
- Han, J., Zhang, J., Zeng, B., Mao, M., 2021. Optimizing dynamic facility location-allocation for agricultural machinery maintenance using benders decomposition. *Omega* 105, 102498.
- Hernández-Pérez, H., Salazar-González, J.J., 2004. A branch-and-cut algorithm for a traveling salesman problem with pickup and delivery. *Discrete Applied Mathematics* 145, 126–139. URL: <https://www.sciencedirect.com/science/article/pii/S0166218X0400071X>, doi:<https://doi.org/10.1016/j.dam.2003.09.013>. graph Optimization IV.
- Hossain, M., 2020. Sharing economy: A comprehensive literature review. *International Journal of Hospitality Management* 87, 102470.

- Jackson, M., Wiktorsson, M., Bellgran, M., 2008. Factory-in-a-box—demonstrating the next generation manufacturing provider, in: Manufacturing Systems and Technologies for the New Frontier: The 41 st CIRP Conference on Manufacturing Systems May 26–28, 2008, Tokyo, Japan, Springer. pp. 341–346.
- Jandyal, A., Chaturvedi, I., Wazir, I., Raina, A., Haq, M.I.U., 2022. 3d printing—a review of processes, materials and applications in industry 4.0. *Sustainable Operations and Computers* 3, 33–42.
- Jiang, P., Leng, J., Ding, K., 2016. Social manufacturing: A survey of the state-of-the-art and future challenges, in: 2016 IEEE International Conference on Service Operations and Logistics, and Informatics (SOLI), IEEE. pp. 12–17.
- Justkowiak, J.E., Pesch, E., 2023. A column generation driven heuristic for order-scheduling and rack-sequencing in robotic mobile fulfillment systems. *Omega* 120, 102897.
- Kantaros, A., Piromalis, D., 2021. Employing a low-cost desktop 3d printer: Challenges, and how to overcome them by tuning key process parameters. *International Journal of Mechanics and Applications* 10, 11–19.
- Kenger, Z.D., Koç, Ç., Özceylan, E., 2021. Integrated disassembly line balancing and routing problem with mobile additive manufacturing. *International Journal of Production Economics* 235, 108088.
- Klarin, A., Suseno, Y., 2021. A state-of-the-art review of the sharing economy: Scientometric mapping of the scholarship. *Journal of Business Research* 126, 250–262.
- Kucukkoc, I., 2019. Milp models to minimise makespan in additive manufacturing machine scheduling problems. *Computers & Operations Research* 105, 58–67.
- Lei, C., Lin, W.H., Miao, L., 2016. A two-stage robust optimization approach for the mobile facility fleet sizing and routing problem under uncertainty. *Computers & Operations Research* 67, 75–89.

- Li, Q., Kucukkoc, I., Zhang, D.Z., 2017. Production planning in additive manufacturing and 3d printing. *Computers & Operations Research* 83, 157–172.
- Liang, Y.J., Luo, Z.X., 2022. A survey of truck–drone routing problem: Literature review and research prospects. *Journal of the Operations Research Society of China* 10, 343–377.
- Liu, W., Liu, L., Qi, X., 2024. Drone resupply with multiple trucks and drones for on-time delivery along given truck routes. *European Journal of Operational Research* 318, 457–468. URL: <https://www.sciencedirect.com/science/article/pii/S0377221724003813>, doi:<https://doi.org/10.1016/j.ejor.2024.05.025>.
- Missaoui, A., Ruiz, R., 2022. A parameter-less iterated greedy method for the hybrid flowshop scheduling problem with setup times and due date windows. *European Journal of Operational Research* 303.
- Mladenović, N., Urošević, D., Hanafi, S., Ilić, A., 2012. A general variable neighborhood search for the one-commodity pickup-and-delivery travelling salesman problem. *European Journal of Operational Research* 220, 270–285. URL: <https://www.sciencedirect.com/science/article/pii/S0377221712000719>, doi:<https://doi.org/10.1016/j.ejor.2012.01.036>.
- Mohr, S., Khan, O., 2015. 3d printing and supply chains of the future, in: *Innovations and Strategies for Logistics and Supply Chains: Technologies, Business Models and Risk Management*. Proceedings of the Hamburg International Conference of Logistics (HICL), Vol. 20, Berlin: epubli GmbH. pp. 147–174.
- Mokotoff, E., 2004. An exact algorithm for the identical parallel machine scheduling problem. *European Journal of Operational Research* 152, 758–769. URL: <https://www.sciencedirect.com/science/article/pii/S0377221702007269>, doi:[https://doi.org/10.1016/S0377-2217\(02\)00726-9](https://doi.org/10.1016/S0377-2217(02)00726-9). applications of Soft O.R. Methods.
- Mont, O., Palgan, Y.V., Bradley, K., Zvolska, L., 2020. A decade of the sharing economy: Concepts, users, business and governance perspectives. *Journal of cleaner production* 269, 122215.

- Nguyen, M.A., Hà, M.H., 2023. The parallel drone scheduling traveling salesman problem with collective drones. *Transportation Science* 57, 866–888. URL: <https://doi.org/10.1287/trsc.2022.1192>, doi:10.1287/trsc.2022.1192, arXiv:<https://doi.org/10.1287/trsc.2022.1192>.
- Ofo, 2014. Ofo sharing bicycles. URL: <https://baike.baidu.com/item/ofo/>. accessed: 2024-07-02.
- Ozturk, O., 2020. A truncated column generation algorithm for the parallel batch scheduling problem to minimize total flow time. *European Journal of Operational Research* 286, 432–443.
- Pasha, J., Dulebenets, M.A., Kavosi, M., Abioye, O.F., Wang, H., Guo, W., 2020. An optimization model and solution algorithms for the vehicle routing problem with a “factory-in-a-box”. *Ieee Access* 8, 134743–134763.
- Ramadhan, F., Irawan, C.A., Salhi, S., Cai, Z., 2025. The truck traveling salesman problem with drone and boat for humanitarian relief distribution in flood disaster: Mathematical model and solution methods. *European Journal of Operational Research* 322, 270–291. URL: <https://www.sciencedirect.com/science/article/pii/S0377221724007975>, doi:<https://doi.org/10.1016/j.ejor.2024.10.022>.
- Shahmoradi-Moghadam, H., Schönberger, J., 2021. Coordinated allocation production routing problem for mobile supply chains with shared factories. *Computers & Chemical Engineering* 155, 107501.
- Shao, K., Fan, W., Lan, S., Kong, M., Yang, S., 2023. A column generation-based heuristic for brachytherapy patient scheduling with multiple treatment sessions considering radioactive source decay and time constraints. *Omega* 118, 102853.
- Shi, Y., Yang, J., Han, Q., Song, H., Guo, H., 2024. Optimal decision-making of post-disaster emergency material scheduling based on helicopter–truck–drone collaboration. *Omega* 127, 103104. URL: <https://www.sciencedirect.com/science/article/pii/S0305048324000707>, doi:<https://doi.org/10.1016/j.omega.2024.103104>.

- Tang, Z., van Hoeve, W.J., 2024. Dual bounds from decision diagram-based route relaxations: An application to truck-drone routing. *Transportation Science* 58, 257–278. URL: <https://doi.org/10.1287/trsc.2021.0170>, doi:10.1287/trsc.2021.0170, arXiv:<https://doi.org/10.1287/trsc.2021.0170>.
- Tracy, T., Wu, L., Liu, X., Cheng, S., Li, X., 2023. 3d printing: Innovative solutions for patients and pharmaceutical industry. *International Journal of Pharmaceutics* 631, 122480.
- Uber, 2024. Go anywhere with uber. URL: <https://www.uber.com/se/sv/>. accessed: 2024-07-02.
- Van Den Akker, J., Hoogeveen, J., van Kempen, J.W., 2006. Parallel machine scheduling through column generation: Minimax objective functions, in: *Algorithms–ESA 2006: 14th Annual European Symposium, Zurich, Switzerland, September 11-13, 2006. Proceedings 14*, Springer. pp. 648–659.
- Wang, F., Li, H., Xiong, H., 2025. Truck–drone routing problem with stochastic demand. *European Journal of Operational Research* 322, 854–869. URL: <https://www.sciencedirect.com/science/article/pii/S0377221724009214>, doi:<https://doi.org/10.1016/j.ejor.2024.11.036>.
- Wang, Y., Ropke, S., Wen, M., Bergh, S., 2023. The mobile production vehicle routing problem: Using 3d printing in last mile distribution. *European Journal of Operational Research* 305, 1407–1423.
- Wiechman, N., Damodaran, P., 2015. A column generation approach for scheduling a batch processing machine with makespan objective. *International Journal of Industrial and Systems Engineering* 21, 334–355.
- Xu, J., Nagi, R., 2013. Identical parallel machine scheduling to minimise makespan and total weighted completion time: a column generation approach. *International Journal of Production Research* 51, 7091–7104.
- Xu, W., Jambhulkar, S., Zhu, Y., Ravichandran, D., Kakarla, M., Vernon, B., Lott, D.G., Cornella, J.L., Shefi, O., Miquelard-Garnier, G., et al., 2021. 3d printing for polymer/particle-based processing: A review. *Composites Part B: Engineering* 223, 109102.

- Zhang, J., Campbell, J.F., Sweeney, D.C., 2024. A continuous approximation approach to integrated truck and drone delivery systems. *Omega* 126, 103067. URL: <https://www.sciencedirect.com/science/article/pii/S0305048324000343>, doi:<https://doi.org/10.1016/j.omega.2024.103067>.
- Zhang, Z., Gong, X., Song, X., Yin, Y., Lev, B., Chen, J., 2022. A column generation-based exact solution method for seru scheduling problems. *Omega* 108, 102581.
- Zhen, L., Gao, J., Wang, S., Laporte, G., Yue, X., 2025. Optimizing an on-demand delivery mode based on trucks and drones. *Transportation Science* .

Appendix A. Experimental results on Instance CL

The CL instance set consists of 12 groups, representing different combinations of $m \in \{2, 3, 4\}$ processing machines and $g \in \{20, 30\}$ jobs, with each group in $n \in \{5, 6\}$ customer nodes. Each group of instances contains 3 test cases, resulting in a total of 36 test cases in the CL instance set. The runtime limit for each test case is set to 7200 seconds. The processing times of jobs on different machines follow a uniform distribution between 1 and 15, while the setup times between jobs follow a uniform distribution between 0 and 20.

The results in Table A.1 indicate that the MILP model fails to find optimal solutions for any instance in the CL set within the given time frame. Furthermore, as the problem size increases, more instances become unsolvable due to memory overflow, with an average of 1.33 instances affected. Additionally, the upper and lower bounds obtained from MILP show a decline in solution quality, failing to produce satisfactory results. In contrast, the LBBD algorithm successfully finds optimal solutions for the first 2 instance sets, with run times under 500 seconds. For the instances which are not solved to optimality by LBBD, the average upper and lower bounds achieved are significantly better than those of MILP.

Focusing on the detailed solution information of the LBBD algorithm, it becomes evident that the increase in problem size is the primary source of computational difficulty. As shown in Table A.2, even the least labels, “LET” and “SL”, average over one million, while the average number of “DL” reaches 19 million. Notably, in instance $\{5, 3, 30\}$, the number of “DL” exceeds 100 million. Additionally, it is observed that when $n = 6$, “Ltime” significantly exceeds “Ctime”, accounting for the majority of the total runtime. This suggests that as the problem size grows, the solution of the MP within the LBBD algorithm becomes the primary bottleneck impacting overall performance.

Appendix B. A Heuristic Method HLBBB

In order to make our LBBD more vigorous for solving large-scale instances, we propose a heuristic method based on LBBD (HLBBB). It is noted that when solving the CL instance set with the LBBD algorithm, excessive focus on solution optimality is highly time-consuming. While in practice, obtaining a solution of satisfactory quality within a shorter time is often acceptable. Therefore, we relax the optimality gap tolerance for

Table A.1: Experimental Results of MILP and LBB on Instance CL

n	m	g	MILP				LBB		
			GAP_{LB}	GAP_{UB}	Time	OFM	GAP_{LB}	GAP_{UB}	Time
5	2	20	83.99	32.51	7200.00	1	0.00	0.00	36.83
5	2	30	92.73	7.74	7200.00	0	0.00	0.00	407.84
5	3	20	75.04	23.47	7200.00	1	0.00	0.00	3020.77
5	3	30	96.24	66.67	7200.00	2	62.14	0.01	7200.00
5	4	20	90.64	66.24	7200.00	2	41.19	0.00	7200.00
5	4	30	94.35	33.33	7200.00	1	46.31	10.84	7200.00
6	2	20	66.41	31.83	7200.00	1	0.00	0.00	3434.85
6	2	30	92.81	32.50	7200.00	1	-0.06	0.06	3132.07
6	3	20	94.89	66.67	7200.00	2	52.56	5.18	7200.00
6	3	30	98.63	56.67	7200.00	2	56.79	0.00	7200.00
6	4	20	86.64	33.33	7200.00	1	44.38	11.77	7200.00
6	4	30	97.12	66.67	7200.00	2	51.87	3.39	7200.00
Average			89.13	43.14	7200.00	1.33	29.60	2.60	5036.03

Table A.2: Impact of acceleration strategies on LBB on Instance CL

n	m	g	TL	LET	SL	DL	ITER	Ltime	Ctime
5	2	20	218419.67	28772.00	11408.00	236617.00	82.00	34.57	2.27
5	2	30	3644239.00	296879.00	837694.33	7437490.00	82.00	32.53	375.31
5	3	20	12975166.00	2481138.33	685804.33	15983933.67	2402.00	2897.20	123.57
5	3	30	46668512.67	5987551.67	8740999.67	111748531.67	781.33	436.46	6763.54
5	4	20	15034878.33	2951118.33	847794.67	17871568.67	3673.00	7007.28	192.72
5	4	30	29846177.00	3720675.33	6052390.67	68105723.00	550.33	325.64	6874.36
6	2	20	231208.00	33570.00	6503.00	156752.33	244.00	3428.37	6.48
6	2	30	1780591.00	280215.00	132066.67	2390148.33	244.00	3115.50	16.58
6	3	20	274861.67	40889.67	6516.00	221440.00	196.67	7193.40	6.60
6	3	30	1797571.67	316472.67	138329.00	2595611.33	191.67	7177.56	22.45
6	4	20	427870.67	75195.67	19760.67	367504.00	200.67	7191.71	8.29
6	4	30	2636086.33	519163.33	209081.67	4342668.67	164.33	7122.17	77.83
Average			9627965.17	1394303.42	1474029.06	19288165.72	734.33	3830.20	1205.83

the MP to ensure a quick feasible MP solution. The optimality gap tolerance defines the allowable difference ratio between the upper bound and the lower bound found during the solving process. When this value is set to 0%, CPLEX will run until it finds the optimal solution. However, when this value is set to a range between 0% and 100%, CPLEX will stop solving once it reaches that value, obtaining the best upper bound currently found. Additionally, we restrict the number of iterations (ITER) to 200 during the LBB algorithm's cycles. As shown in Table B.1, $GAP_{LB}(\%) = \frac{UB^* - LB'}{UB^*} \times 100\%$, $GAP_{UB}(\%) = \frac{UB' - UB^*}{UB^*} \times 100\%$ where LB' and UB' represent the lower bound and upper bound obtained by the current algorithm, and UB^* represents the best upper bound obtained by LBB and HLBB. The “MPGAP(%)” denotes the optimality gap tolerance for the MP. In LBB, the MPGAP is set to 0% for all cases, while in HLBB, we set it to different values of 50%, 70%, and 90% based on the size of the instances. The larger the MPGAP, the less accurate the solution is, but the run time will be significantly reduced. Due to the relaxed lower bound of the MP not being a valid lower bound, we only compare the upper bound (GAP_{UB}) of HLBB. “NUM” represents the number of instances where the HLBB achieved the best upper bound. For example, 5.00(1) in Table B.3 means the HLBB can get the best upper bound in 5 instances, and in 1 of those, the HLBB finds the best upper bound while the LBB can not.

Although this heuristic method (HLBB) can not guarantee the optimality of solutions, it consistently achieve high-quality solutions. The results of LBB and HLBB on the 5 instance sets C2, C3, C4, CL, and Set A are presented in Tables B.1 to B.5, respectively. As shown in Table B.1, HLBB achieves an average iteration count of 3.84, which is less than half of LBB (8.50), while maintaining an average GAP_{UB} of only 2.82%. For most instances (average NUM = 7.38), HLBB can find the optimal solution with shorter run times. In Table B.2, except for instance {3, 4, 15}, the GAP_{UB} for the remaining instances in HLBB is below 14%, with an average GAP_{UB} of 9.99%. In the instance {3, 4, 15}, LBB cannot find the optimal solution within the specified 7200s, whereas HLBB can quickly obtain a feasible upper bound in under 700s. As the instance size increases, the advantage of HLBB in run time becomes more pronounced. In Tables B.3 and B.4, the average run times of LBB reach 4148.44s and 5036.03s, respectively, while these of HLBB are only 26.47s and 337.53s, with average GAP_{UB} s below 11%. Notably, for some instances among {{4,4,5}, {5,3,30}, {5,4,20}},

$\{6,3,20\}$, $\{6,3,30\}$, $\{6,4,30\}$, the values of NUM indicate that HLBBB can find a better upper bound than LBBB. Finally, we compared the performances of HLBBB and LBBB on Set A, revealing that the heuristic HLBBB algorithm can achieve an acceptable upper bound of 6.21% within a short run time of 136.26s, while most instances can also be solved optimally.

Therefore, we can conclude that the heuristic algorithm based on LBBB can obtain high-quality solutions in a shorter time when solving large-scale instances.

Table B.1: Comparisons of LBBB and HLBBB on Instance C2

n	m	g	LBBB				HLBBB				
			GAP_{LB}	GAP_{UB}	Time	ITER	GAP_{UB}	Time	ITER	MPGAP	NUM
2	2	5	0.00	0.00	0.06	4.00	2.03	0.03	3.20	50.00	9.00
2	2	8	0.00	0.00	4.71	4.00	3.43	0.99	3.50	50.00	3.00
2	2	10	0.00	0.00	255.08	4.00	4.59	21.86	3.70	50.00	2.00
2	3	5	0.00	0.00	0.11	8.00	8.42	0.06	3.30	50.00	8.00
2	3	8	0.00	0.00	18.97	8.00	0.11	7.40	4.50	50.00	9.00
2	3	10	0.00	0.00	150.54	8.00	0.12	132.07	4.00	50.00	9.00
2	4	5	0.00	0.00	0.79	16.00	3.88	0.55	5.50	50.00	9.00
2	4	8	0.00	0.00	84.58	16.00	0.00	77.43	3.00	50.00	10.00
Average			0.00	0.00	64.36	8.50	2.82	30.05	3.84	50.00	7.38

Table B.2: Comparisons of LBBD and HLBBB on Instance C3

n	m	g	LBBD				HLBBB				
			GAP_{LB}	GAP_{UB}	Time	ITER	GAP_{UB}	Time	ITER	MPGAP	NUM
3	2	5	0.00	0.00	0.54	26.20	1.93	0.39	18.00	50.00	6.00
3	2	8	0.00	0.00	1.24	28.40	3.91	1.33	22.20	50.00	3.00
3	2	10	0.00	0.00	0.93	27.00	3.72	0.63	20.40	50.00	4.00
3	2	15	0.00	0.00	51.03	30.60	3.11	27.94	21.10	70.00	6.00
3	3	5	0.00	0.00	3.28	126.60	8.36	1.17	42.60	50.00	2.00
3	3	8	0.00	0.00	4.23	135.30	6.90	1.50	55.80	50.00	2.00
3	3	10	0.00	0.00	8.67	139.80	8.29	2.60	61.50	50.00	3.00
3	3	15	0.00	0.00	779.13	141.90	18.59	89.08	36.10	70.00	1.00
3	4	5	0.00	0.00	30.41	509.80	11.86	0.70	29.50	50.00	0.00
3	4	8	0.00	0.00	58.21	588.60	13.56	1.81	55.10	50.00	0.00
3	4	10	0.00	0.00	173.83	588.60	13.53	10.42	82.20	50.00	0.00
3	4	15	44.56	0.00	7200.00	268.40	26.12	622.30	36.50	70.00	1.00
Average			3.71	0.00	692.63	217.60	9.99	63.32	40.08	55.00	2.33

Table B.3: Comparisons of LBBD and HLBBB on Instance C4

n	m	g	LBBD				HLBBB				
			GAP_{LB}	GAP_{UB}	Time	ITER	GAP_{UB}	Time	ITER	MPGAP	NUM
4	2	5	0.00	0.00	54.04	340.20	0.64	25.75	197.40	50.00	9.00
4	2	8	0.00	0.00	73.36	429.00	4.33	25.60	200.00	50.00	5.00
4	2	10	0.00	0.00	73.51	417.40	2.46	25.82	200.00	50.00	5.00
4	2	15	0.00	0.00	87.41	471.00	1.18	24.89	200.00	50.00	5.00
4	3	5	2.26	0.00	4041.65	3573.30	4.02	24.85	186.60	50.00	3.00
4	3	8	0.92	0.00	4750.96	3992.90	13.53	25.64	200.00	50.00	0.00
4	3	10	1.77	0.00	5253.86	4133.30	16.39	27.28	200.00	50.00	0.00
4	3	15	8.25	0.00	6646.46	4503.10	19.79	27.83	200.00	50.00	0.00
4	4	5	33.07	1.80	7200.00	6403.90	1.29	24.24	200.00	50.00	6.00(2)
4	4	8	36.54	0.00	7200.00	6457.90	9.80	27.30	200.00	50.00	1.00
4	4	10	37.75	0.00	7200.00	6295.00	11.95	29.76	200.00	50.00	1.00
4	4	15	38.42	0.00	7200.00	6058.10	22.03	28.67	200.00	50.00	0.00
Average			13.25	0.15	4148.44	3589.59	8.95	26.47	198.67	50.00	2.92

Table B.4: Comparisons of LBB and HLBB on Instance CL

n	m	g	LBB				HLBB				
			GAP_{LB}	GAP_{UB}	Time	ITER	GAP_{UB}	Time	ITER	MPGAP	NUM
5	2	20	0.00	0.00	36.83	82.00	0.40	16.15	75.67	90.00	2.00
5	2	30	0.00	0.00	407.84	82.00	2.71	248.89	62.00	90.00	1.00
5	3	20	0.00	0.00	3020.77	2402.00	27.27	21.83	123.67	90.00	0.00
5	3	30	61.51	1.84	7200.00	781.33	2.57	953.98	200.00	90.00	1.00(1)
5	4	20	39.98	2.16	7200.00	3673.00	6.90	39.75	200.00	90.00	2.00(2)
5	4	30	51.54	0.00	7200.00	550.33	21.48	1222.88	200.00	90.00	0.00
6	2	20	0.00	0.00	3434.85	244.00	5.03	259.09	200.00	90.00	0.00
6	2	30	0.00	0.00	3132.07	244.00	5.83	257.72	200.00	90.00	0.00
6	3	20	53.57	2.94	7200.00	196.67	8.99	197.64	200.00	90.00	2.00(2)
6	3	30	56.41	1.03	7200.00	191.67	16.63	152.12	200.00	90.00	1.00(1)
6	4	20	49.60	0.00	7200.00	200.67	16.10	299.04	200.00	90.00	0.00
6	4	30	53.23	0.31	7200.00	164.33	13.91	381.25	200.00	90.00	1.00(1)
Average			30.49	0.69	5036.03	734.33	10.65	337.53	171.78	90.00	0.83

Table B.5: Comparisons of LBB and HLBB on Set A

Index	n	m	g	LBB				HLBB				
				GAP_{LB}	GAP_{UB}	Time	ITER	GAP_{UB}	Time	ITER	MPGAP	NUM
A01	3	2	10	0.00	0.00	0.63	10.00	0.00	0.79	10.00	50.00	1.00
A02	3	2	10	0.00	0.00	0.92	10.00	0.00	0.66	10.00	50.00	1.00
A03	3	2	5	0.00	0.00	0.31	10.00	0.00	0.34	9.00	50.00	1.00
A04	3	3	5	0.00	0.00	1.88	50.00	22.57	0.38	12.00	50.00	0.00
A05	3	3	10	0.00	0.00	5.30	50.00	8.35	0.89	18.00	50.00	0.00
A06	3	3	10	0.00	0.00	3.64	50.00	0.00	4.02	50.00	50.00	1.00
A07	3	3	10	0.00	0.00	2.61	50.00	0.00	3.33	50.00	50.00	1.00
A08	3	3	10	0.00	0.00	2.79	50.00	0.00	2.68	50.00	50.00	1.00
A09	3	2	15	0.00	0.00	39.41	10.00	0.00	31.45	10.00	50.00	1.00
A10	3	3	15	0.00	0.00	1423.41	50.00	0.00	1443.17	50.00	50.00	1.00
A11	3	2	15	0.00	0.00	35.15	10.00	0.00	30.12	10.00	50.00	1.00
A12	3	3	15	0.00	0.00	1389.53	50.00	22.43	302.22	22.00	50.00	0.00
A13	4	2	15	0.00	0.00	3.47	28.00	0.00	2.96	28.00	50.00	1.00
A14	5	3	20	52.26	0.00	7200.00	2402.00	33.60	84.58	200.00	50.00	0.00
Average				3.73	0.00	722.07	202.14	6.21	136.26	37.79	50.00	0.71

Appendix C. Detailed experimental results

This section presents the upper bounds obtained by the LBBD algorithm for each case in the C2, C3, C4, CL, and A instance. For clarity, the upper bound values are rounded to the nearest integer, while the run time is reported to one decimal place.

Table C.1: The UB of LBBD on Instance C2

n	m	g	Idx	UB	Time	UB	Time	UB	Time	UB	Time	UB	Time
2	2	5	1-5	67	0.1	47	0.1	37	0.1	90	0.1	52	0.0
			6-10	63	0.0	72	0.1	82	0.0	69	0.1	69	0.0
2	2	8	1-5	64	7.0	86	6.8	78	1.8	66	2.7	96	4.8
			6-10	95	2.7	85	3.5	68	3.7	87	5.5	75	8.7
2	2	10	1-5	89	123.8	101	397.4	93	394.9	84	151.2	69	175.6
			6-10	69	336.9	96	303.7	95	220.0	89	327.6	81	119.7
2	3	5	1-5	30	0.1	66	0.2	66	0.1	64	0.1	54	0.1
			6-10	63	0.1	73	0.1	80	0.1	72	0.1	72	0.1
2	3	8	1-5	34	11.9	83	13.0	78	7.5	74	6.8	60	7.7
			6-10	94	113.3	64	6.4	45	9.5	61	4.7	52	8.9
2	3	10	1-5	80	181.2	61	98.7	82	105.1	52	147.6	79	218.9
			6-10	82	158.1	61	173.7	87	166.2	77	90.8	97	165.0
2	4	5	1-5	44	0.7	27	0.9	54	0.8	70	0.9	60	0.7
			6-10	42	0.5	35	0.6	59	0.8	76	1.1	59	0.9
2	4	8	1-5	70	68.8	71	87.4	44	73.4	55	191.9	52	55.9
			6-10	61	37.5	63	78.9	69	70.2	70	115.9	81	65.8

Table C.2: The UBs of LBB on Instance C3

n	m	g	Idx	UB	Time	UB	Time	UB	Time	UB	Time	UB	Time
3	2	5	1-5	151	0.6	79	0.5	106	0.6	71	0.4	75	0.6
			6-10	97	0.4	137	0.5	116	0.6	70	0.7	102	0.4
3	2	8	1-5	101	1.4	104	0.9	97	0.9	131	1.5	96	2.5
			6-10	106	1.7	132	1.7	125	0.9	128	0.4	121	0.6
3	2	10	1-5	145	0.7	120	0.6	166	1.0	112	0.8	160	0.6
			6-10	160	1.6	102	1.3	130	1.1	168	0.8	96	0.9
3	2	15	1-5	110	165.2	112	41.8	184	40.5	153	52.3	177	32.6
			6-10	104	35.8	150	40.1	134	32.5	116	40.7	132	28.7
3	3	5	1-5	27	4.6	111	2.3	100	3.1	114	3.0	102	3.1
			6-10	136	3.1	120	3.7	105	3.6	60	3.2	98	3.1
3	3	8	1-5	99	4.6	91	3.5	82	5.0	121	4.2	116	4.3
			6-10	70	3.7	70	3.6	66	4.5	96	3.5	63	5.4
3	3	10	1-5	79	5.5	94	22.1	108	7.8	108	7.6	137	5.7
			6-10	118	8.0	83	7.3	78	8.6	140	6.1	96	8.0
3	3	15	1-5	116	429.4	174	882.9	96	1023.7	123	672.0	141	976.2
			6-10	99	678.8	129	685.5	133	795.3	121	770.1	85	877.4
3	4	5	1-5	108	35.7	85	36.7	144	24.6	82	33.5	73	31.6
			6-10	103	21.7	99	32.6	130	21.0	92	33.6	87	33.2
3	4	8	1-5	136	29.5	101	38.9	69	79.9	126	70.9	84	57.4
			6-10	107	54.8	116	59.6	90	56.1	61	74.8	86	60.2
3	4	10	1-5	105	141.5	98	149.2	79	143.9	140	180.7	106	112.4
			6-10	106	169.8	95	129.7	85	163.5	105	131.2	65	416.4
3	4	15	1-5	144	7200.0	159	7200.0	128	7200.0	96	7200.0	126	7200.0
			6-10	109	7200.0	94	7200.0	108	7200.0	98	7200.0	74	7200.0

Table C.3: The UBs of LBB on Instance C4

n	m	g	Idx	UB	Time	UB	Time	UB	Time	UB	Time	UB	Time
4	2	5	1-5	140	46.1	165	51.1	150	48.6	106	31.9	107	48.8
			6-10	158	42.5	121	66.1	116	54.6	131	50.3	91	100.3
4	2	8	1-5	110	78.1	135	79.2	147	47.8	151	81.5	160	50.6
			6-10	80	78.5	111	78.5	98	67.1	82	85.1	124	87.3
4	2	10	1-5	177	61.2	167	80.3	124	71.0	135	101.3	167	65.0
			6-10	113	83.9	98	82.2	167	58.3	160	52.4	122	79.6
4	2	15	1-5	137	103.1	161	101.6	163	97.9	185	93.2	192	70.7
			6-10	190	75.6	118	82.9	156	83.9	123	85.0	153	80.1
4	3	5	1-5	105	3537.9	131	2844.1	89	7200.0	172	953.5	109	1199.7
			6-10	110	5256.8	120	5498.6	127	3564.6	80	7200.0	115	3155.6
4	3	8	1-5	128	7200.0	111	7200.0	144	5572.4	140	5712.7	121	2697.0
			6-10	126	2690.6	113	4740.8	118	3074.8	96	1415.6	131	7200.0
4	3	10	1-5	122	6125.3	191	6095.0	90	4041.4	161	4520.8	111	5302.1
			6-10	117	4047.1	83	7200.0	152	4443.8	128	5108.2	123	5653.6
4	3	15	1-5	96	7200.0	129	6218.1	170	7200.0	154	7200.0	105	7200.0
			6-10	138	7200.0	171	7200.0	119	5993.6	147	3839.5	105	7200.0
4	4	5	1-5	69	7200.0	129	7200.0	102	7200.0	132	7200.0	168	7200.0
			6-10	110	7200.0	133	7200.0	125	7200.0	99	7200.0	149	7200.0
4	4	8	1-5	59	7200.0	128	7200.0	114	7200.0	138	7200.0	131	7200.0
			6-10	122	7200.0	117	7200.0	108	7200.0	122	7200.0	90	7200.0
4	4	10	1-5	135	7200.0	122	7200.0	167	7200.0	140	7200.0	109	7200.0
			6-10	117	7200.0	135	7200.0	146	7200.0	117	7200.0	129	7200.0
4	4	15	1-5	108	7200.0	146	7200.0	122	7200.0	137	7200.0	132	7200.0
			6-10	131	7200.0	145	7200.0	144	7200.0	183	7200.0	168	7200.0

Table C.4: The UBs of LBBD on Instance CL

n	m	g	Idx	UB	Time	UB	Time	UB	Time
5	2	20	1-3	219	40.0	207	34.7	223	35.8
5	2	30	1-3	248	348.0	233	218.1	275	657.4
5	3	20	1-3	155	3157.9	201	3177.2	182	2727.2
5	3	30	1-3	232	7200.0	222	7200.0	205	7200.0
5	4	20	1-3	211	7200.0	156	7200.0	181	7200.0
5	4	30	1-3	282	7200.0	245	7200.0	209	7200.0
6	2	20	1-3	223	3998.5	231	3712.0	211	2594.0
6	2	30	1-3	266	1979.8	292	4240.7	255	3175.7
6	3	20	1-3	211	7200.0	187	7200.0	153	7200.0
6	3	30	1-3	210	7200.0	240	7200.0	234	7200.0
6	4	20	1-3	179	7200.0	163	7200.0	205	7200.0
6	4	30	1-3	233	7200.0	260	7200.0	291	7200.0

Table C.5: The UBs of LBBD on Set A

Idx	n	m	g	UB	Time
A01	3	2	10	143	0.6
A02	3	2	10	151	0.9
A03	3	2	5	127	0.3
A04	3	3	5	230	1.9
A05	3	3	10	263	5.3
A06	3	3	10	111	3.6
A07	3	3	10	116	2.6
A08	3	3	10	90	2.8
A09	3	2	15	227	39.4
A10	3	3	15	121	1423.4
A11	3	2	15	183	35.1
A12	3	3	15	245	1389.5
A13	4	2	15	230	3.5
A14	5	3	20	295	7200.0

Appendix D. Proof of Dominance 1

Given two labels $\mathcal{L}^1$ and $\mathcal{L}^2$ ending at the same job g , and satisfying the dominance rule. Let $\mathcal{L}^3$ and $\mathcal{L}^4$ be two labels extended from $\mathcal{L}^1$ and $\mathcal{L}^2$ to job h , respectively. If labels $\mathcal{L}^3$ and $\mathcal{L}^4$ satisfy the dominance conditions, then we can prove the validity of the dominance rule according to mathematical induction.

If extending to an unused machine, the parameters associated with labels $\mathcal{L}^3$ and $\mathcal{L}^4$ have the following relations:

- From $c'(\mathcal{L}^1) \leq c'(\mathcal{L}^2)$, we have

$$c'(\mathcal{L}^3) = r_h + c'(\mathcal{L}^1) - r_g - \lambda_h \alpha_{l,h} \leq r_h + c'(\mathcal{L}^2) - r_g - \lambda_h \alpha_{l,h} = c'(\mathcal{L}^4);$$
- From $r(\mathcal{L}^1) \leq r(\mathcal{L}^2)$, we have $r(\mathcal{L}^3) = \max\{r(\mathcal{L}^1), e_{0,h} + p_{h,\pi_h}\} \leq \max\{r(\mathcal{L}^2), e_{0,h} + p_{h,\pi_h}\} = r(\mathcal{L}^4);$
- From $\eta(\mathcal{L}^1) = \eta(\mathcal{L}^2)$, we have $\eta(\mathcal{L}^3) = h = \eta(\mathcal{L}^4);$
- From $\tau(\mathcal{L}^1) \leq \tau(\mathcal{L}^2)$, we have $\tau(\mathcal{L}^3) = e_{0,h} + p_{h,\pi_h} = \tau(\mathcal{L}^4);$
- From $\pi(\mathcal{L}^1) = \pi(\mathcal{L}^2)$, we have $\pi(\mathcal{L}^3) = \Delta_g^1 = \pi(\mathcal{L}^4);$
- From $N(\mathcal{L}^1) \supseteq N(\mathcal{L}^2)$, since $N(\mathcal{L}^3) = N(\mathcal{L}^1) \cup h$, $N(\mathcal{L}^4) = N(\mathcal{L}^2) \cup h$, it is easy to get $N(\mathcal{L}^3) \supseteq N(\mathcal{L}^4);$
- From $\Delta(\mathcal{L}^1) \supseteq \Delta(\mathcal{L}^2)$, we have $\Delta(\mathcal{L}^3) = \Delta(\mathcal{L}^1) \setminus \Delta_g^1$, $\Delta(\mathcal{L}^4) = \Delta(\mathcal{L}^2) \setminus \Delta_g^1$, then we can get $\Delta(\mathcal{L}^3) \supseteq \Delta(\mathcal{L}^4).$

If extending to a job, the parameters associated with labels $\mathcal{L}^3$ and $\mathcal{L}^4$ have the following relations:

- From $c'(\mathcal{L}^1) \leq c'(\mathcal{L}^2)$, we have

$$c'(\mathcal{L}^3) = r_h + c'(\mathcal{L}^1) - r_g - \lambda_h \alpha_{l,h} \leq r_h + c'(\mathcal{L}^2) - r_g - \lambda_h \alpha_{l,h} = c'(\mathcal{L}^4);$$
- From $r(\mathcal{L}^1) \leq r(\mathcal{L}^2)$, we have $r(\mathcal{L}^3) = \max\{r(\mathcal{L}^1), \tau(\mathcal{L}^1) + e_{g,h} + p_{h,\pi_h}\} \leq \max\{r(\mathcal{L}^2), \tau(\mathcal{L}^2) + e_{g,h} + p_{h,\pi_h}\} = r(\mathcal{L}^4);$
- From $\eta(\mathcal{L}^1) = \eta(\mathcal{L}^2)$, we have $\eta(\mathcal{L}^3) = h = \eta(\mathcal{L}^4);$

- From $\tau(\mathcal{L}^1) \leq \tau(\mathcal{L}^2)$, we have $\tau(\mathcal{L}^3) = \tau(\mathcal{L}^1) + e_{g,h} + p_{h,\pi_h} \leq \tau(\mathcal{L}^2) + e_{g,h} + p_{h,\pi_h} = \tau(\mathcal{L}^4)$;
- From $\pi(\mathcal{L}^1) = \pi(\mathcal{L}^2)$, we have $\pi(\mathcal{L}^3) = \pi(\mathcal{L}^1) = \pi(\mathcal{L}^2) = \pi(\mathcal{L}^4)$;
- From $N(\mathcal{L}^1) \supseteq N(\mathcal{L}^2)$, since $N(\mathcal{L}^3) = N(\mathcal{L}^1) \cup h$, $N(\mathcal{L}^4) = N(\mathcal{L}^2) \cup h$, it is easy to get $N(\mathcal{L}^3) \supseteq N(\mathcal{L}^4)$;
- From $\Delta(\mathcal{L}^1) \supseteq \Delta(\mathcal{L}^2)$, we have $\Delta(\mathcal{L}^3) = \Delta(\mathcal{L}^1)$, $\Delta(\mathcal{L}^4) = \Delta(\mathcal{L}^2)$, then we can get $\Delta(\mathcal{L}^3) \supseteq \Delta(\mathcal{L}^4)$.

Obviously, the reduced cost of label $\mathcal{L}^3$ is smaller than or equal to that of label $\mathcal{L}^4$. The dominance rule is valid for our algorithm.

Appendix E. Comparison with the enumeration approach

To highlight the computational time advantage of tandem sequence-based column, we have compared it with the enumeration method (ENU). In the LBBD approach, SPs are solved using tandem sequence-based column generation; in contrast, the enumeration method enumerates all possible allocation and scheduling sequences of machines and jobs included in each SP one by one. We report the comparison of computational times between LBBD and ENU in Tables E.1-E.4, where "ENU" represents the enumeration method and "—" indicates that the enumeration fails to complete within the time limit (7200 seconds). Since ENU could not finish the enumeration within the time limit for all test cases in instance set CL, we only compared ENU with LBBD on the instance sets C2, C3, C4, and Set A.

As shown in Table E.1, ENU fails to complete the enumeration within the time limit for the instances $\{2,2,10\}$, $\{2,3,8\}$, $\{2,3,10\}$, and $\{2,4,8\}$ on instance C2. Among the remaining instances, LBBD outperforms ENU in computational time on instances $\{2,2,5\}$ and $\{2,3,5\}$, though the differences between these two methods are not significant. For instances $\{2,2,8\}$ and $\{2,4,5\}$, the computational time of LBBD is within 5 seconds, while the maximum computational time of ENU is close to 600 seconds.

Table E.2 presents a comparison between LBBD and ENU on the instance C3. Similarly, ENU fails to complete the solution process within the time limit for the instances $\{3,2,15\}$, $\{3,3,15\}$, $\{3,4,10\}$, and $\{3,4,15\}$. LBBD and ENU perform very closely on the instance $\{3,2,8\}$, with LBBD being only

Table E.1: Computational Time of LBBD and ENU on Instance C2

n	m	g	LBBD			ENU	
			GAP_{LB}	GAP_{UB}	Time	GAP_{UB}	Time
2	2	5	0.00	0.00	0.06	0.00	0.13
2	2	8	0.00	0.00	4.71	0.00	591.78
2	2	10	0.00	0.00	255.08	—	—
2	3	5	0.00	0.00	0.11	0.00	1.69
2	3	8	0.00	0.00	18.97	—	—
2	3	10	0.00	0.00	150.54	—	—
2	4	5	0.00	0.00	0.79	0.00	101.38
2	4	8	0.00	0.00	84.58	—	—

Table E.2: Computational Time of LBBD and ENU on Instance C3

n	m	g	LBBD			ENU	
			GAP_{LB}	GAP_{UB}	Time	GAP_{UB}	Time
3	2	5	0.00	0.00	0.54	0.00	0.74
3	2	8	0.00	0.00	1.24	0.00	1.27
3	2	10	0.00	0.00	0.93	0.00	5.02
3	2	15	0.00	0.00	51.03	—	—
3	3	5	0.00	0.00	3.28	0.00	3.62
3	3	8	0.00	0.00	4.23	0.00	16.32
3	3	10	0.00	0.00	8.67	0.00	829.83
3	3	15	0.00	0.00	779.13	—	—
3	4	5	0.00	0.00	30.41	0.00	50.00
3	4	8	0.00	0.00	58.21	0.00	400.65
3	4	10	0.00	0.00	173.83	—	—
3	4	15	42.91	3.21	7200.00	—	—

0.03 seconds faster than ENU. However, overall, LBBD is comprehensively superior to ENU in terms of computational time on instance C3. In fact, for the specific instance $(\{3,3,10\})$, the computational time of ENU is nearly 100 times that of LBBD.

Table E.3: Computational Time of LBBD and ENU on Instance C4

n	m	g	LBBD			ENU	
			GAP_{LB}	GAP_{UB}	Time	GAP_{UB}	Time
4	2	5	0.00	0.00	54.04	0.00	62.32
4	2	8	0.00	0.00	73.36	0.00	169.15
4	2	10	0.00	0.00	73.51	0.00	205.01
4	2	15	0.00	0.00	87.41	0.00	238.38
4	3	5	2.26	0.00	4041.65	—	—
4	3	8	0.92	0.00	4750.96	—	—
4	3	10	1.77	0.00	5253.86	—	—
4	3	15	8.25	0.00	6646.46	—	—
4	4	5	34.26	0.00	7200.00	—	—
4	4	8	36.54	0.00	7200.00	—	—
4	4	10	37.75	0.00	7200.00	—	—
4	4	15	38.42	0.00	7200.00	—	—

Tables E.3 and E.4 present the comparison results between LBBD and ENU on instance C4 and Set A, respectively. For instance C4, ENU can only solve 4 instances: $\{4,2,5\}$, $\{4,2,8\}$, $\{4,2,10\}$, and $\{4,2,15\}$, with computational times all slower than those of LBBD. All other instances failed to complete the solution within the time limit. In Set A, ENU can not finish solving instances A09, A10, A11, A12, and A14 within the time limit. For the remaining instances, the computational times of ENU are significantly slower than those of LBBD.

Table E.4: Computational Time of LBBD and ENU on Set A

Index	n	m	g	LBBD			EUN	
				GAP_{LB}	GAP_{UB}	Time	GAP_{UB}	Time
A01	3	2	10	0.00	0.00	0.63	0.00	2.24
A02	3	2	10	0.00	0.00	0.92	0.00	1.82
A03	3	2	5	0.00	0.00	0.31	0.00	1.66
A04	3	3	5	0.00	0.00	1.88	0.00	6.13
A05	3	3	10	0.00	0.00	5.30	0.00	41.45
A06	3	3	10	0.00	0.00	3.64	0.00	30.69
A07	3	3	10	0.00	0.00	2.61	0.00	27.66
A08	3	3	10	0.00	0.00	2.79	0.00	28.34
A09	3	2	15	0.00	0.00	39.41	—	—
A10	3	3	15	0.00	0.00	1423.41	—	—
A11	3	2	15	0.00	0.00	35.15	—	—
A12	3	3	15	0.00	0.00	1389.53	—	—
A13	4	2	15	0.00	0.00	3.47	0.00	87.60
A14	5	3	20	48.24	8.44	7200.00	—	—

Highlights

Logic-based Benders Decomposition Combined with Column Generation for Mobile 3D Printer Scheduling Problem

Tao Li, Hu Qin, Nan Huang

- A Mobile 3D Printer Scheduling Problem with Setup times is proposed.
- A three-index model is formulated, and the logic-based Benders decomposition is applied.
- A tandem-sequence-based column generation and 3 strategies are developed to solve the sub-problem
- Four instance sets were generated to evaluate the algorithm's efficiency.

**Logic-based Benders Decomposition Combined with Column Generation for
Mobile 3D Printer Scheduling Problem**

(Original Title: Tandem-Sequence-Based Column Generation for Mobile 3D Printer
Scheduling Problem)

by

Tao Li ^a, Hu Qin^a, Nan Huang^{b, *}

^aFirst Author: Tao Li

School of Management
Huazhong University of Science and Technology
No. 1037 Luoyu Road
Wuhan, China
Mail: litao.hust@qq.com

^aSecond Author: Hu Qin

School of Management
Huazhong University of Science and Technology
No. 1037 Luoyu Road
Wuhan, China
Mail: tigerqin@hust.edu.cn

^{b, *}Correspondence: Nan Huang

College of Science and Engineering
Hamad Bin Khalifa University
Qatar Foundation,
Doha, Qatar
Mail: nan_huang@hotmail.com

There is no conflict of interest.

The author declared no potential conflict of interest with respect to the research, authorship, and/or publication of this article.